

Tesi Mattia Prestifilippo

From Token-Based to Expertise-Based Governance: Verifiable Credentials for Decentralized Autonomous Organizations

1. Introduction

Negli ultimi anni, le **Decentralized Autonomous Organizations (DAOs)** hanno acquisito una crescente rilevanza nell'ecosistema blockchain, e vengono utilizzate per coordinare comunità di utenti e gestire quantità significative di capitale condiviso tra i membri di tali organizzazioni. Gli attuali sistemi di governance assegnano generalmente ai partecipanti un potere di voto proporzionale alla quantità di **governance token** posseduti. In un sistema di questo tipo, maggiore è lo stake depositato da un membro, maggiore sarà la sua influenza sulle decisioni della DAO.

Questo approccio presenta tuttavia una criticità: la quantità di capitale posseduta da un partecipante non rappresenta necessariamente la sua capacità di valutare correttamente una determinata proposta. Un membro che possiede un numero elevato di token può infatti esercitare una forte influenza sulla governance indipendentemente dalle proprie competenze rispetto all'argomento oggetto della decisione. Di conseguenza, un partecipante può influenzare significativamente l'esito di una votazione pur non possedendo competenze specifiche nell'ambito trattato dalla proposta.

Un'ulteriore criticità dei sistemi di governance basati esclusivamente sui token riguarda la **concentrazione del potere decisionale**. Quando una parte significativa dei governance token è detenuta da un numero ristretto di partecipanti, questi ultimi possono esercitare un'influenza sproporzionata sulle decisioni della DAO. Tale situazione può ridurre l'effettiva decentralizzazione del processo decisionale, poiché i membri con una quantità limitata di token, anche qualora possiedano competenze rilevanti rispetto alla proposta, possono avere un'influenza marginale sull'esito della votazione, mentre i membri che hanno la maggior parte dei token controllano di fatto le decisioni prese dalla DAO, **centralizzando** il potere decisionale.

La presente tesi propone quindi un modello di **competence-based governance**, nel quale il potere di voto di un membro non dipende esclusivamente dallo stake detenuto, ma viene determinato combinando il contributo economico del partecipante con le sue **competenze verificabili** relative all'argomento trattato dalla proposta.

Seguendo il paradigma della **Self-Sovereign Identity (SSI)**, tali competenze vengono certificate da un **issuer** attraverso il rilascio di **Verifiable Credentials (VCs)**. Le credenziali permettono al membro di dimostrare il possesso di determinate competenze senza basarsi su semplici dichiarazioni personali. Prima che tali competenze possano contribuire al calcolo del potere di voto, le Verifiable Credentials vengono **verificate** dalla governance, consentendo alla DAO di utilizzare informazioni certificate e verificabili direttamente all'interno del proprio processo decisionale.

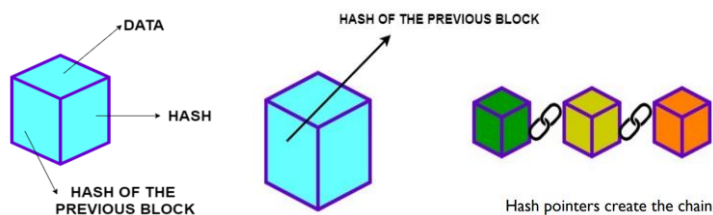
2. Background

2.1 Blockchain and Smart Contracts

2.1.1 Definizione di Blockchain

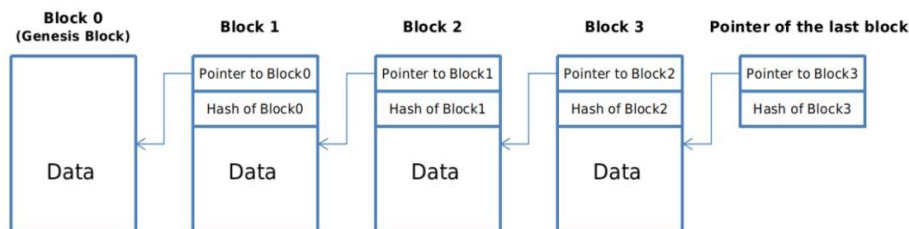
Un **ledger** è un **registro** mantenuto in una rete di nodi distribuiti che tiene traccia di una serie ordinata di eventi registrati. Può essere solo **Append-only**, ovvero è possibile soltanto aggiungere nuovi eventi **senza mai modificarli**.

Cash				
Date	Description	Increase	Decrease	Balance
Jan. 1, 20X3	Balance forward			\$ 50,000
Jan. 2, 20X3	Collected receivable	\$ 10,000		60,000
Jan. 3, 20X3	Cash sale	5,000		65,000
Jan. 5, 20X3	Paid rent		\$ 7,000	58,000
Jan. 7, 20X3	Paid salary		3,000	55,000
Jan. 8, 20X3	Cash sale	4,000		59,000
Jan. 8, 20X3	Paid bills		2,000	57,000
Jan. 10, 20X3	Paid tax		1,000	56,000
Jan. 12, 20X3	Collected receivable	7,000		63,000



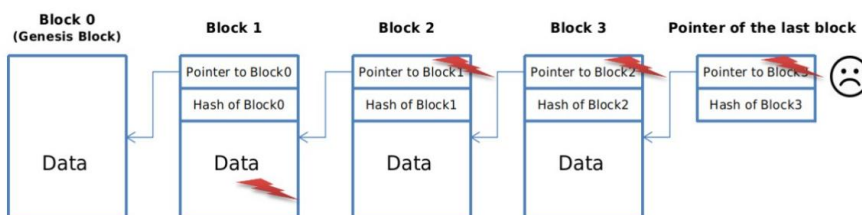
Una **blockchain** è un **registro** distribuito e immutabile, composto da una **lista di blocchi** collegati tra loro tramite puntatori hash.

Ogni **blocco** della catena contiene le **transazioni** effettuate dalla rete e l'**header del blocco** contenente i vari metadata, un valore casuale **nonce** e l'**hash del blocco precedente**, che rappresenta un **riferimento** univoco a tale blocco.



La blockchain è mantenuta da una rete di **nodi distribuiti**, che mantengono ognuno la stessa **copia completa e aggiornata** del registro.

Se un attaccante modifica i dati in un blocco **k**, il suo **hash cambia**, e quindi l'hash pointer contenuto nel blocco successivo **k+1** non corrisponde più a quello precedente. Questo crea un effetto a catena: Il blocco **k+1** non ha più un **hash al blocco precedente valido**. Di conseguenza, anche il blocco **k+2** avrà un **hash pointer errato** e così via. Alla fine, l'intera blockchain risulterà **compromessa**, e la modifica sarà facilmente rilevabile.



Per rendere possibile una modifica malevola l'attaccante deve modificare tutti gli hash pointer successivi e per ogni blocco successivo ricalcolare la Proof of Work.

2.1.2 Consenso: Proof of Work e Proof of Stake

In una blockchain il **consenso** è il processo attraverso il quale i nodi della rete si accordano su **quali transazioni** o blocchi siano **validi e in quale ordine** devono essere inseriti nel registro condiviso.

Nella blockchain Bitcoin il consenso è basato su Proof of Work, mentre su Ethereum è basata su Proof of Stake.

Nella soluzione basata su **Proof of Work**, invece di votare in base a quanti nodi partecipano, il consenso viene determinato in base alla **quantità di potenza computazionale** che ogni nodo **mette a disposizione**. Anche se un attaccante registra più identità, queste non aggiungono potenza di calcolo, quindi non può votare più volte. Viene definito un **puzzle crittografico** costoso in termini di potenza computazionale. Un nodo **miner** deve trovare tramite **bruteforce** un valore **casuale** x chiamato **nonce**, tale che la concatenazione tra **l'hash** dell'**header del blocco creato** e il valore casuale **nonce** sia minore ad una soglia di difficoltà T , $H(\text{header}||x) < T$.

Il nodo che per primo risolve il problema, diventa "leader" della scelta del blocco, e "vince" il diritto di **decidere unilateralmente** quale transazioni (valide) inserire nel prossimo blocco della blockchain e in quale ordine. Inoltre, riceve una **ricompensa** monetaria per il suo lavoro. Ogni nodo della rete, una volta ricevuto il blocco, in modo rapido calcola l'hash del block header ricevuto e **controlla** che questo hash sia **inferiore** al target di difficoltà stabilito. Controlla inoltre che le transazioni al suo interno siano valide. Se non valide, il blocco non viene accettato. Se il blocco supera la verifica, ogni nodo lo **aggiunge** alla propria copia della blockchain, aggiornando il registro delle transazioni.

La **Proof of Stake (PoS)** si basa invece sul capitale, espresso in ETH, che i partecipanti mettono in **stake** come garanzia del proprio comportamento.

I partecipanti al consenso sono chiamati **validatori**. Per attivare direttamente un validatore Ethereum è necessario depositare **32 ETH** come stake. Questo deposito svolge la funzione di garanzia: un validatore che rispetta correttamente il protocollo può ricevere ricompense, mentre comportamenti scorretti possono causare penalità e, nei casi più gravi, la perdita di parte dello stake. Il tempo in Ethereum è suddiviso in intervalli chiamati **slot**, ciascuno della durata di circa 12 secondi. Un insieme di 32 slot costituisce un **epoch**. In ogni slot, il protocollo seleziona un validatore come **block proposer**, incaricato di costruire un nuovo blocco contenente le transazioni da aggiungere alla blockchain.

Una volta ricevuto il blocco proposto, gli altri validatori verificano che il blocco e le transazioni contenute al suo interno rispettino le regole del protocollo. Se ritengono valida la proposta, producono una **attestation**, cioè un voto crittograficamente firmato con cui indicano quale blocco considerano corretto e quale catena ritengono essere quella valida.

(Parte da mergiare con appunti P2P).

2.1.2 Ethereum: Blockchain Ethereum, Smart Contract, Transactions, Token ERC20 e Gas

Ethereum è una blockchain che viene utilizzata non solo come registro di transazioni distribuito, ma come esecutore di programmi su nodi distribuiti sulla propria rete.

Ethereum mantiene uno **stato globale condiviso** che può essere modificato attraverso transazioni e tramite l'esecuzione di programmi chiamati **smart contract**.

La criptovaluta nativa della rete è **Ether (ETH)**, che viene utilizzata per trasferire valore, partecipare al meccanismo di consenso Proof of Stake e pagare il gas relativo alle operazioni eseguite sulla rete.

Ogni operazione computazionale richiede infatti una determinata quantità di **gas**, che rappresenta la quantità di risorse necessarie per eseguirla su tutti i nodi della rete.

L'esecuzione del codice avviene su tutti i nodi attraverso la **Ethereum Virtual Machine (EVM)**, che permette ai nodi Ethereum di eseguire gli smart contract in modo deterministico, in modo che partendo dallo stesso stato e dalla stessa transazione, i nodi possono ottenere lo stesso risultato e concordare sul nuovo stato della blockchain.

Uno **smart contract** è un programma memorizzato sulla blockchain Ethereum che contiene **regole e funzioni** eseguite quando un utente o un altro smart contract interagisce con esso.

Uno smart contract viene normalmente scritto in un linguaggio come **Solidity** e successivamente **deployato** sulla blockchain. Dopo il deployment, possiede un proprio **indirizzo**, attraverso il quale può essere utilizzato.

Gli utenti interagiscono con lo smart contract attraverso **transazioni** o chiamate alle sue funzioni. Se una funzione modifica i dati del contratto, la modifica viene verificata dalla rete Ethereum e registrata sulla blockchain.

L'esecuzione delle operazioni richiede il pagamento di **gas**, che dipende dalle risorse computazionali utilizzate.

La caratteristica principale degli smart contract è che le regole definite nel codice vengono applicate **automaticamente**, senza la necessità di un intermediario centrale.

Il **gas** in Ethereum è l'unità utilizzata per misurare la quantità di lavoro computazionale necessaria per eseguire una transazione o una funzione di uno smart contract. Ogni operazione eseguita dalla **EVM** ha un determinato costo in gas.

Il gas serve principalmente ad impedire che gli utenti utilizzino gratuitamente le risorse computazionali della rete e proteggere Ethereum da operazioni eccessivamente costose o da cicli di esecuzione infiniti. Per questo motivo, chi invia una transazione paga una **gas fee** in ETH.

Il costo finale di una transazione dipende dalla quantità di gas effettivamente utilizzata e dal prezzo pagato per ogni unità di gas. Si ha: $\text{Transaction Fee} = \text{Gas Used} \times \text{Price per Gas}$

Il prezzo del gas viene normalmente espresso in **gwei**, dove 1 gwei corrisponde a $10^{(-9)}$ ETH.

Ogni transazione specifica inoltre un **gas limit**, cioè il massimo quantitativo di gas che può essere utilizzato per la sua esecuzione. Se l'esecuzione richiede più gas del limite disponibile, la transazione fallisce; il gas già consumato per l'esecuzione non viene comunque restituito.

Una **transazione Ethereum** è un'operazione inviata da un account alla rete per **modificare lo stato della blockchain**. Una transazione può essere usata, ad esempio, per trasferire **ETH** da un account a un altro, eseguire una funzione di uno **smart contract**, o effettuare il deployment di un nuovo smart contract.

Ogni transazione viene creata e **firmata digitalmente** dal mittente utilizzando la propria chiave privata. La firma permette alla rete di verificare che la transazione sia stata realmente autorizzata dal proprietario dell'account.

Una transazione contiene diverse informazioni, tra cui l'account **sender** che invia la transazione, l'account **recipient** destinatario, la quantità di ETH **value** eventualmente trasferita, le informazioni aggiuntive **data**, il numero progressivo **nonce** associato alle transazioni dell'account, e la quantità massima **gas limit** che il mittente è disposto a spendere.

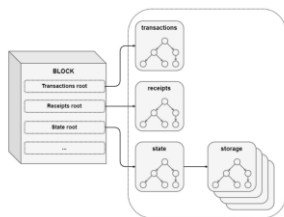
2.1.3 Storage Model di Ethereum

Ethereum mantiene uno **stato globale**, chiamato **world state**, che contiene le informazioni associate a tutti gli account Ethereum. Ogni account Ethereum è rappresentato da un **nonce**, che tiene traccia delle operazioni associate all'account, un saldo **balance** in ETH, uno **storageRoot**, che nel caso di uno smart contract rappresenta la **merkle root** dello storage persistente del contratto, e l'hash del codice dello smart contract **codeHash**.

Quando una transazione viene eseguita, può modificare lo stato globale di Ethereum. Il risultato dell'esecuzione produce quindi un nuovo stato globale.

Oltre allo stato globale, ogni blocco contiene anche un insieme di transazioni e le relative **transaction receipts**. Tali ricevute contengono informazioni sul risultato dell'esecuzione di una transazione, come lo stato di successo o fallimento, il gas utilizzato e gli eventuali eventi generati dagli smart contract.

Ethereum per ogni blocco utilizza **tre** principali **root crittografiche** associate per memorizzare lo stato di Ethereum. Si ha una **stateRoot**, che rappresenta la **merkle root** dello **world state** risultante dopo l'esecuzione del blocco. Si ha una **transactionsRoot**, che rappresenta la merkle root delle transazioni incluse nel blocco. Si ha una **receiptsRoot**, che rappresenta la merkle root delle ricevute prodotte dall'esecuzione delle transazioni.



Riguardo lo storage effettivo dei dati usati per l'esecuzione degli smart contract, vengono salvati nel seguente modo dalle EVM presenti sui nodi. Il bytecode viene salvato sotto forma di hash nel world state del blocco Ethereum, mentre il codice effettivo viene salvato in un database presente sul nodo su cui gira la EVM. La stessa cosa accade per lo storage persistente del contratto, per cui la merkle root dei dati viene salvata nel world state, mentre i dati effettivi vengono salvati in un database interni al nodo. I dati temporanei vengono salvati in una memoria volatile, analoga alla RAM, che viene azzerata al termine dell'esecuzione del contratto. In particolare si ha uno **stack LIFO** in cui vengono spinti gli operandi delle istruzioni e da cui vengono poi letti per calcolare risultati intermedi, un buffer lineare di byte chiamato **Memory** contenente dati temporanei e risultati di calcolo, e un'area read-only chiamata **Calldata** contenente i dati in input inviati dal chiamante del contratto.

2.1.4 Solidity

Solidity è un linguaggio di programmazione ad alto livello e staticamente tipizzato utilizzato per sviluppare **smart contract** compatibili con la **EVM**. Il codice Solidity viene compilato in bytecode, successivamente deployato sulla blockchain ed eseguito dalla EVM.

Attraverso Solidity è possibile definire **variabili di stato**, **strutture dati**, **funzioni** ed **eventi** che determinano il comportamento di uno smart contract.

Le variabili di stato permettono di memorizzare dati persistenti nello storage del contratto, mentre le funzioni definiscono le operazioni che possono essere eseguite dagli utenti o da altri smart contract.

Solidity mette a disposizione diversi **tipi di dato**, tra cui interi (uint, int), valori booleani (bool), indirizzi Ethereum (address), array e tipi definiti dall'utente.

Tra le strutture dati più utilizzate vi sono i **mapping**, che permettono di associare una chiave a un valore, ad esempio per associare a ciascun indirizzo Ethereum il proprio saldo depositato.

Le funzioni in Solidity possono avere diversi livelli di **visibilità**, che stabiliscono **da dove possono essere chiamate**. Nel caso una funzione sia **public**, può essere chiamata sia dall'interno del contratto sia dall'esterno, nel caso **external**, può essere chiamata dall'esterno del contratto, nel caso **internal** può essere chiamata solo dal contratto stesso e dai contratti che lo ereditano, nel caso **private** può essere chiamata solo all'interno del contratto in cui è definita.

Le funzioni possono inoltre essere dichiarate **view** quando **leggono lo stato del contratto senza modificarlo**.

I **modifiers** sono costrutti che permettono di aggiungere condizioni prima dell'esecuzione di una funzione. Ad esempio, un modifier può verificare che chi richiama una funzione sia un indirizzo autorizzato, e se la condizione non è soddisfatta, l'esecuzione viene bloccata.

Gli **eventi** permettono invece a uno smart contract di registrare informazioni nei **log della blockchain**. Questi log possono essere letti da applicazioni esterne per sapere quando è avvenuta una determinata operazione.

Solidity supporta inoltre **inheritance**, **interfaces** e **libraries**, che permettono di organizzare e riutilizzare meglio il codice.

L'**inheritance** permette a uno smart contract di **ereditare variabili, funzioni e comportamenti da un altro contratto**, evitando di riscrivere codice già esistente. Il contratto derivato può aggiungere nuove funzionalità o modificare alcuni comportamenti del contratto di base.

Le **interfaces** definiscono invece **quali funzioni un contratto mette a disposizione**, senza specificarne necessariamente l'implementazione.

Le **libraries** sono raccolte di funzioni riutilizzabili che possono essere usate da più smart contract per evitare di riscrivere la stessa logica. Può contenere, ad esempio, funzioni per eseguire calcoli, manipolare strutture dati o applicare operazioni comuni. Queste funzioni vengono poi copiate inline all'interno dei contratti che le utilizzano.

2.1.5 Token Fungible ERC20

I **token ERC-20** sono token **fungibili** implementati sulla blockchain Ethereum, ossia asset digitali tali che ogni unità è equivalente e intercambiabile con un'altra unità dello stesso tipo. Ad esempio, 1 token ABC possiede le stesse caratteristiche di qualsiasi altro token ABC.

Lo standard ERC-20 definisce un insieme di **funzioni e regole comuni** che uno smart contract deve implementare per gestire i token in modo coerente.

Le funzioni principali sono **totalSupply()**, che restituisce la quantità totale di token esistenti, **balanceOf(address)**, che restituisce il saldo di token posseduto da un indirizzo, **transfer(address, amount)**, che trasferisce token da un account a un altro, **approve(address, amount)**, che delega un altro indirizzo a prelevare una certa quantità dei propri token, **allowance(owner, spender)**, che indica quanti token un indirizzo spender è autorizzato a prelevare, e **transferFrom(from, to, amount)**, che permette a un soggetto autorizzato via **approve()** di trasferire token dall'indirizzo from all'indirizzo to.

Un contratto ERC-20 mantiene una **tabella di balance**, rappresentata da un mapping tra gli **indirizzi** Ethereum e il numero di **token posseduti** da ciascun indirizzo. Ad esempio, si ha [Alice → 100 ERC-20 token, Bob → 50 ERC-20 token, Carol → 25 ERC-20 token].

Il contratto mantiene anche una **tabella di allowance**, rappresentata da un mapping che associa un indirizzo holder proprietario dei token, ad un indirizzo allowed autorizzato a prelevare i token, ad un intero allowance che rappresenta il numero massimo di token che il delegato può ancora prelevare. Una data riga viene aggiornata quando vengono chiamate le funzioni **approve()** e **transferFrom()**.

I token possono essere creati attraverso un'operazione chiamata **minting**, che aumenta la **totalSupply**, e possono eventualmente essere distrutti tramite **burning**, che riduce la **totalSupply**.

I token ERC-20 possono essere utilizzati anche come **governance token**, cioè token che attribuiscono ai membri di una DAO un determinato **potere di voto**.

Lo standard ERC-20, però, registra principalmente il **saldo corrente** di token posseduto da ciascun account e non permette, da solo, di conoscere quanti token un utente possedeva in un dato momento precedente.

Nei sistemi di governance è invece importante poter determinare il **potere di voto posseduto in uno specifico momento**, ad esempio quando viene creata una proposta. Per questo motivo possono essere utilizzate estensioni come **ERC20Votes**, che aggiungono meccanismi di **checkpoint**, attraverso i quali è possibile registrare e recuperare il voting power storico di un account.

2.2 Decentralized Autonomous Organizations

Una **Decentralized Autonomous Organization** è una organizzazione digitale composta da più utenti membri, che non ha una leadership centrale, ma viene governata dai suoi membri in modo decentralizzato.

Le regole, la gestione dei fondi e i processi decisionali non sono affidati ad attori centrali esterni, ma ad uno smart contract eseguito su una blockchain.

Una DAO **nasce** quando viene deployato il suo **smart contract** sulla blockchain. Tale smart contract sostituisce gli statuti contenenti le regole di gestione dell'organizzazione. Definisce come si entra nella DAO, come vengono spesi i fondi e come funziona il processo decisionale.

Una DAO si dice **autonoma** perché una volta lanciata, **può funzionare senza intervento umano diretto**, finché la blockchain su cui viene eseguito lo smart contract di governance continua a esistere.

Una DAO si dice **decentralizzata** perché non c'è un CEO o un consiglio di amministrazione che governa l'organizzazione, ma il potere è distribuito tra i membri, in modo proporzionale ai fondi che depositano nella DAO.

All'interno delle DAO, per quantificare il potere di voto che un membro possiede, si utilizzano dei token di governance. Quando un membro deposita dei fondi nella DAO, riceve proporzionalmente dei token di governance. I

I membri che hanno determinati requisiti possono creare una **proposal**, che propone agli altri membri di fare eseguire all'organizzazione una determinata azione, come spendere i fondi della DAO, cambiare una regola di governance, o inserire un nuovo membro.

I membri votano usando i loro **token di governance**. Ogni token rappresenta una quota di voto, e vince la proposta che ottiene la maggioranza dei voti. Il voto è **trasparente e verificabile** sulla blockchain. Una volta chiuso il voto, il risultato è **vincolante**.

Se una **proposta passa**, lo smart contract **esegue automaticamente** la decisione.

Ad esempio, uno scenario possibile può essere rappresentato da un gruppo di persone che crea una DAO per investire insieme in startup. I membri che investono fondi ricevono proporzionalmente governance token. Tali fondi investiti sono bloccati in uno smart contract, in modo tale che possono essere prelevati solo tramite una proposta accettata di governance. In questo scenario, si ha un membro che propone di investire 100 ETH nella startup X. I membri della DAO votano utilizzando i propri token. Se la proposta ottiene la maggioranza dei voti, lo smart contract invia i soldi alla startup X.

2.2.1 Caratteristiche di una DAO

Il deposito e il trasferimento del valore avvengono sotto forma di **criptovalute**.

Una DAO ha la capacità di organizzare e archiviare le votazioni.

Le funzionalità offerte da una DAO e le sue regole sono codificate su una blockchain, mediante uno smart contract.

Il potere decisionale non è nelle mani di un singolo attore centrale che possa prendere tutte le decisioni per conto dell'organizzazione.

Le **decisioni** vengono prese in modo completamente **autonomo** e **decentralizzato** dai **membri** dell'organizzazione.

Le proposte, votazioni e azioni effettuate dalla DAO sono **trasparenti** su blockchain, insieme allo smart contract che ne definisce la governance.

Il raggiungimento del **consenso** viene preso da parte tramite **votazioni basate su regole predefinite**, come vittoria per maggioranza, per superquorum, e validità delle votazioni per raggiungimento di quorum.

Il **processo decisionale** inizia sempre con una **proposal**. Una **proposal** è una proposta formale inserita nel sistema di governance, che descrive una decisione da prendere, e le azioni da eseguire se viene approvata dalla maggioranza dei membri della DAO. Per poter effettuare una proposta, un membro deve superare un **Proposal Threshold**, ossia un **requisito minimo** necessario per **poter creare** una **proposta**, come ad esempio possedere almeno una certa quantità di token.

I **diritti di voto** sono assegnati ad ogni membro in base al numero di **token posseduti**. Il **quorum** è la **quantità minima di partecipazione** richiesta affinché una votazione sia considerata valida.

Un membro della DAO può delegare a se stesso o ad un altro membro i propri token, trasferendo il proprio potere di voto al delegato, senza perdere la proprietà dei token delegati.

2.2.2 Meccanismi di governance

La **governance** di una DAO è l'insieme di regole e meccanismi attraverso cui i membri partecipano alle decisioni dell'organizzazione. Definisce chi può **proporre** una **decisione**, chi può **votare**, quanto **pesa** ciascun **voto**, quali **condizioni** devono essere soddisfatte affinché una proposta sia approvata, e come la decisione viene eseguita.

Il **Governance Contract** è lo smart contract che implementa le regole della governance, ossia implementa la creazione delle proposte, la gestione del voto, il conteggio dei risultati e l'approvazione o rifiuto finale della proposta.

Nel processo decisionale generalmente viene effettuata una proposta, in cui un membro fornisce una descrizione, le azioni da eseguire in caso di successo, la durata della votazione, la soglia di vittoria, e il quorum di validità.

I membri votano la proposta. Si ha un **voting period**, ossia un intervallo di tempo durante il quale gli aventi diritto possono votare su una proposta prima che venga chiusa e conteggiata. Finito il voting period, vengono conteggiati i voti ricevuti dalla proposta. Se viene raggiunto un quorum, la proposta è valida. Se la proposta supera la soglia di voti affermativa, diventa **Approved**. In tal caso vengono eseguite le azioni prestabilite.

Se non supera il quorum o la soglia di voti, diventa **Rejected**.

L'**esecuzione** è la fase in cui una proposta approvata dalla governance viene effettivamente applicata, eseguendo le operazioni previste tramite smart contract.

L'**esecuzione** può essere **automatica** on-chain, facendo partire automaticamente un'azione prevista dallo smart contract, oppure **manuale** off-chain, in cui dopo che la proposta è stata approvata, l'azione **non viene eseguita automaticamente dallo smart contract**, ma richiede l'intervento di soggetti esterni, che la realizzano manualmente, come ad esempio inviare fondi manualmente, chiamare una funzione su un contratto o eseguire attività nel mondo reale.

2.2.3 Meccanismo di Timelock

Il **Timelock** è un contratto che impone un **ritardo obbligatorio** tra la proposta approvata e l'esecuzione effettiva dell'azione.

Dopo che una proposta è **approvata**, l'azione viene **messa in coda** nel Timelock. Deve aspettare un tempo preimpostato, prima di essere eseguita dal Timelock.

Questo permette alla community di accorgersi di proposte malevole e dà tempo per reagire.

2.2.4 Snapshot

Uno **snapshot** è un meccanismo con cui la DAO fotografa il potere di voto dei partecipanti in un preciso istante, tipicamente **al blocco numero X** oppure a un timestamp specifico.

Al momento del conteggio dei voti, non viene usato il saldo di token del momento in cui un partecipante vota, ma il saldo del momento temporale in cui la proposta viene creata.

Se un partecipante sposta token, o compra token solo per votare, in modo da influenzare le votazioni, in un momento successivo alla creazione della proposta, questo viene reso inutile.

2.3 Self-Sovereign Identity

2.3.1 Digital Identity centralizzata

Nei sistemi di **digital identity centralizzati**, le informazioni relative all'identità di un utente vengono generalmente gestite da un **soggetto centrale esterno**, come ad esempio un ente pubblico o un'università. Questo soggetto ha il compito di registrare i dati dell'utente, verificarne l'identità e, quando necessario, confermare le informazioni relative all'utente a soggetti terzi.

Il problema di questo approccio è che i dati dell'utente sono gestiti da un'unica autorità, dalla quale l'utente dipende per conservarli, aggiornarli e dimostrarli a terzi.

Questo riduce il controllo diretto dell'utente sui propri dati e introduce un **single point of failure**. Infatti, se il sistema centrale diventa indisponibile, anche l'accesso e la verifica delle identità possono essere compromessi.

Ad esempio, se una laurea è verificabile solo tramite il database dell'università, lo studente dipende continuamente dall'università per dimostrare il proprio titolo.

Usando questi approccio, per dimostrare un singolo attributo può essere necessario fornire più dati del necessario. Ad esempio, per dimostrare di avere più di 18 anni può essere richiesto un documento che mostra anche nome, indirizzo e data completa di nascita.

Inoltre, le informazioni conservate in una determinata piattaforma possono essere difficili da utilizzare in un'altra.

2.3.2 Definition and Principles

La **Self-Sovereign Identity (SSI)** è un modello di gestione dell'identità digitale in cui **l'individuo è l'unico proprietario** dei propri dati identificativi, mantenendo il controllo diretto delle proprie informazioni, senza dover dipendere da un'autorità centrale che conservi e gestisca i propri dati.

Nel modello SSI l'utente riceve da un'entità fidata delle credenziali che attestano i dati dell'utente. L'utente conserva autonomamente le proprie credenziali in un wallet digitale, e decide **quali informazioni condividere, con chi e per quanto tempo**.

Nel modello SSI, l'identità deve esistere **indipendentemente da autorità centrali**.

L'utente deve avere **pieno controllo sui propri dati personali**, decidendo cosa condividere e con chi. L'utente deve poter **accedere sempre ai propri dati**.

L'identità deve essere **duratura nel tempo**, non legata alla vita di una singola piattaforma.

Le credenziali devono essere **trasferibili tra sistemi diversi**, dando la possibilità di usare la stessa identità su più piattaforme.

La condivisione dei dati deve avvenire **solo con consenso esplicito dell'utente**. Devono essere condivisi **solo i dati strettamente necessari** per il servizio richiesto. Ad esempio, se per entrare in un locale serve sapere se hai più di 18 anni, non serve condividere nome, indirizzo, data completa di nascita. I diritti dell'utente devono prevalere sugli interessi delle organizzazioni. La privacy è centrale, non opzionale.

2.3.3 SSI Actors

Nel modello SSI si hanno tre attori principali: Si ha un **Issuer**, che è il soggetto che certifica un'informazione relativa all'utente e rilascia una **Verifiable Credential**, firmata digitalmente con la propria chiave privata. Per esempio, un'università può certificare che una persona possiede una laurea in Computer Science. Si ha un **Holder**, che è il soggetto che riceve la credenziale dall'issuer, la conserva e può successivamente decidere di presentarla a un Verifier.

Si ha un **Verifier**, che è il soggetto che riceve la credential e ne verifica la validità, controllando che sia stata realmente rilasciata da un **Issuer** considerato affidabile e che il suo contenuto non sia stato modificato.

Ad esempio, un'università rilascia a uno studente una credenziale digitale che certifica il conseguimento di una laurea magistrale in Computer Science. L'università è l'**issuer**, lo studente è l'**holder**. Successivamente, lo studente vuole dimostrare questa competenza a una DAO. Presenta quindi la propria credenziale alla DAO, che assume il ruolo di **verifier** e verifica che la credenziale non sia stata modificata e sia stata rilasciata da un issuer considerato affidabile.

2.3.4 Verifiable Credentials

Le **Verifiable Credentials (VCs)** sono credenziali digitali che permettono a un soggetto **issuer** di attestare in modo verificabile un insieme di **informazioni** relative a un altro soggetto **holder**. Tali informazioni sono rappresentate attraverso dei **claim**, cioè affermazioni riguardanti **attributi** relativi al soggetto della credenziale.

Ad esempio, un'università può rilasciare una Verifiable Credential che attesta che uno studente ha conseguito una laurea magistrale in Computer Science. Si ha come claim il nominativo del soggetto, la facoltà, il voto di laurea e la data di conseguimento.

Una VC contiene sia le **informazioni** certificate dall'issuer, sia i **metadati** necessari a verificare crittograficamente che la credenziale provenga realmente dall'issuer dichiarato e che il suo contenuto non sia stato modificato dopo l'emissione.

Una Verifiable Credential può, ad esempio, contenere informazioni quali l'identità dell'issuer, il tipo di credenziale, il soggetto a cui i claim si riferiscono, gli attributi certificati, l'eventuale periodo di validità, e un riferimento per verificarne lo stato, ossia se è valida, scaduta o revocata.

Il processo inizia con l'**emissione della credenziale**. L'issuer crea una VC contenente i claim che intende certificare e utilizza un meccanismo crittografico basato su hash e firma digitale per poter verificare successivamente provenienza e integrità della credenziale. La credenziale

viene quindi consegnata all'**holder**, che può conservarla e successivamente utilizzarla per dimostrare le informazioni contenute al suo interno.

Quando il titolare desidera utilizzare una credenziale, può presentarla al soggetto interessato a verificare le informazioni certificate, ossia il **verifier**.

Il **verifier** quando riceve la credenziale, controlla quindi che una credential non sia stata alterata e che sia stata effettivamente emessa dal soggetto dichiarato.

Questo, tuttavia, non implica automaticamente che il verifier debba considerare affidabile quell'issuer. Ad esempio, una credenziale che certifica una laurea può essere tecnicamente autentica, ma il verifier può decidere di accettarla solamente se è stata rilasciata da un'università riconosciuta.

Un esempio di VC è quello di un'università che rilascia ad Alice una Verifiable Credential che attesta il conseguimento di una laurea magistrale in Computer Science. L'università agisce come **issuer**, Alice come **holder** e conserva la credenziale nel proprio wallet digitale. Quando Alice deve dimostrare la propria qualifica a un'organizzazione, può presentare la credenziale al relativo **verifier**. Quest'ultimo verifica che la credenziale sia autentica, che non sia stata modificata, che sia ancora valida e che provenga da un issuer ritenuto affidabile. In questo modo Alice può riutilizzare una certificazione digitale verificabile senza richiedere all'università una nuova attestazione ogni volta che deve dimostrare la stessa qualifica.

2.3.5 Struttura di una Verifiable Credential

Una **VC** è composta da un insieme di informazioni che descrivono **chi ha emesso la credenziale, il soggetto a cui si riferisce, quali informazioni certifica e per quanto tempo è valida**. Il W3C definisce alcuni attribut standard che possono essere utilizzati all'interno di una VC.

Si ha un **Issuer** che identifica il soggetto che ha emesso la credenziale. Si ha un **Credential Subject** che contiene le informazioni relative al soggetto. All'interno di questa sezione vengono inseriti i **claim**, cioè le informazioni certificate dall'issuer.

Si ha un campo **Type**, che indica il tipo di credenziale, ad esempio se si tratta di una credenziale universitaria, professionale o di un determinato tipo.

Si ha un **periodo di validità** tramite cui è possibile indicare da quale momento e fino a quando la credenziale deve essere considerata valida.

Si ha delle **informazioni per la verifica crittografica**, tramite cui una VC viene protetta attraverso un meccanismo crittografico che permette al verifier di controllarne l'autenticità e l'integrità.

```

{
  "@context": [
    "https://www.w3.org/2018/credentials/v1",
    "https://www.w3.org/2018/credentials/examples/v1"
  ],
  "type": "VerifiablePresentation",
  "verifiableCredential": [{
    "@context": [
      "https://www.w3.org/2018/credentials/v1",
      "https://www.w3.org/2018/credentials/examples/v1"
    ],
    "id": "http://example.edu/credentials/1872",
    "type": ["VerifiableCredential", "AlumniCredential"],
    "issuer": "https://example.edu/issuers/565049",
    "issuanceDate": "2010-01-01T19:23:24Z",
    "credentialSubject": {
      "id": "did:example:ebfeb1f712ebc6f1c276e12ec21",
      "alumniOf": {
        "id": "did:example:c276e12ec21ebfeb1f712ebc6f1",
        "name": [{"value": "Example University", "lang": "en"},
                  {"value": "Exemple d'Université", "lang": "fr"}]
      }
    }
  }],
  "proof": {
    "type": "RsaSignature2018",
    "created": "2017-06-18T21:19:10Z",
    "proofPurpose": "assertionMethod",
    "verificationMethod": "https://example.edu/issuers/565049#key-1",
    "jws": "eyJhbGciOiJSUzI1NiIsImI2NCI6ZmFsc2UsImNyaXQiOlsiYjY0..."
  }
},
{
  "type": "RsaSignature2018",
  "created": "2018-09-14T21:19:10Z",
  "proofPurpose": "authentication",
  "verificationMethod": "did:example:ebfeb1f712ebc6f1c276e12ec21#keys-1",
  "challenge": "1f44d55f-f161-4938-a659-f8026467f126",
  "domain": "4jt78h47fh47",
  "jws": "eyJhbGciOiJSUzI1NiIsImI2NCI6ZmFsc2UsImNyaXQiOlsiYjY0..."
}
}

```

2.3.6 Verifiable Presentation

Una **Verifiable Presentation (VP)** è una presentazione digitale creata dal **holder** per fornire a un **verifier** una o più informazioni provenienti dalle proprie Verifiable Credentials. Una VP può contenere dati derivati da una singola VC oppure combinare informazioni provenienti da più credenziali. Il suo scopo è permettere al holder di presentare al verifier le informazioni necessarie per una determinata verifica, senza dover necessariamente consegnare tutte le credenziali possedute. Le VP sono generalmente create per una specifica interazione e non sono pensate come credenziali permanenti da conservare nel tempo.

Ad esempio, Alice possiede nel proprio wallet una Verifiable Credential relativa alla laurea e una seconda credenziale che certifica la conoscenza della lingua inglese. Per candidarsi a un lavoro che richiede entrambe le qualifiche, Alice può creare una Verifiable Presentation contenente le informazioni necessarie provenienti dalle due credenziali e presentarla all'azienda, che agisce come verifier.

2.3.7 Selective Disclosure

La **Selective Disclosure** è un meccanismo che permette al holder di una Verifiable Credential di **rivelare al verifier solamente una parte delle informazioni contenute nella credenziale**, evitando di condividere dati che non sono necessari per la verifica richiesta. Ad esempio, una credenziale digitale può contenere nome, cognome, data di nascita, indirizzo, nazionalità, ed età. Se il verifier deve solamente controllare che il soggetto abbia più di 18 anni, non è necessario mostrare tutte queste informazioni. L'holder può presentare solamente l'età, senza rivelare gli altri attributi contenuti nella credenziale.

2.4 Hash Functions

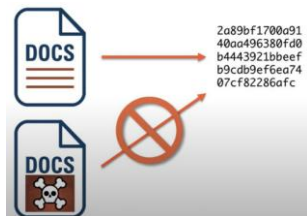
Funzione Hash: Una **funzione hash** è un algoritmo che trasforma un **input** di lunghezza arbitrario n in un **valore hash** di **lunghezza fissa**.

Proprietà fondamentali di una funzione hash: Lo stesso input deve sempre **produrre lo stesso output**. Il calcolo dell'hash deve essere **veloce**. I valori hash devono essere **distribuiti in modo uniforme** nel codominio. La **probabilità** che due input diversi producano lo **stesso hash** deve essere **bassa**.



Funzione Hash Crittografica: Funzione hash che rende impossibile **trovare collisioni** o **determinare l'input originale** a partire dal valore hash.

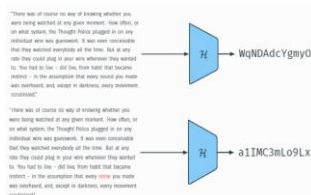
Resistenza alle collisioni: Dato un valore di input x_1 , deve essere computazionalmente **difficile** trovare un **altro valore** di input x_2 **distinto** tale che $H(x_1)=H(x_2)$.



Resistenza alla preimmagine: Dato un **output** h , deve essere computazionalmente **impossibile risalire** al corrispettivo valore x di **input** tale che $H(x)=h$, se non usando il bruteforce.



Avalanche effect: Se nello stesso documento viene cambiato anche un **singolo bit** il valore hash sarà completamente **differente**.



Proprietà di Hiding: Se un **valore segreto** è combinato con un **valore casuale** e poi hashato, l'output non deve fornire informazioni sul **valore segreto**.

Proprietà di Puzzle-friendliness: Deve essere impossibile trovare un input che produce un **hash** con **particolari proprietà** senza **provare molte combinazioni**.

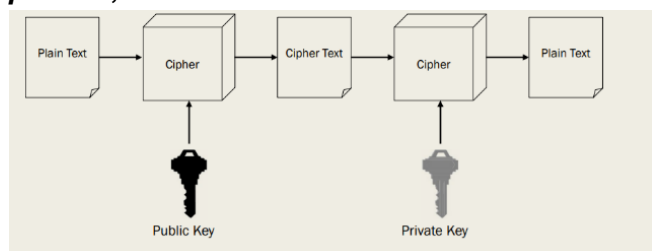
Principio dei piccioni: Se si hanno più elementi rispetto ai contenitori disponibili, almeno un contenitore conterrà più elementi. Poiché il numero di input è molto maggiore rispetto al numero di output possibili nelle funzioni hash, è inevitabile che almeno due input diversi vengano mappati allo stesso hash.

2.5 Crittografia asimmetrica

La **crittografia asimmetrica** utilizza **chiavi diverse** per **crittografare e decrittografare** i messaggi. Ogni utente ha una chiave **pubblica** e una chiave **privata**. I messaggi possono essere crittografati utilizzando una delle due chiavi, ma possono essere decrittografati solo con l'altra chiave.

Le chiavi pubbliche possono essere **pubblicate** dal proprietario della chiave e non è necessario trasmettere una chiave in modo sicuro.

Per garantire la **confidenzialità**, il mittente crittografa un messaggio utilizzando la **chiave pubblica** del **destinatario**. Il destinatario decrittografa il messaggio utilizzando la **sua chiave privata**, che **solo lui conosce**. Solo tale destinatario può decrittografare il messaggio.

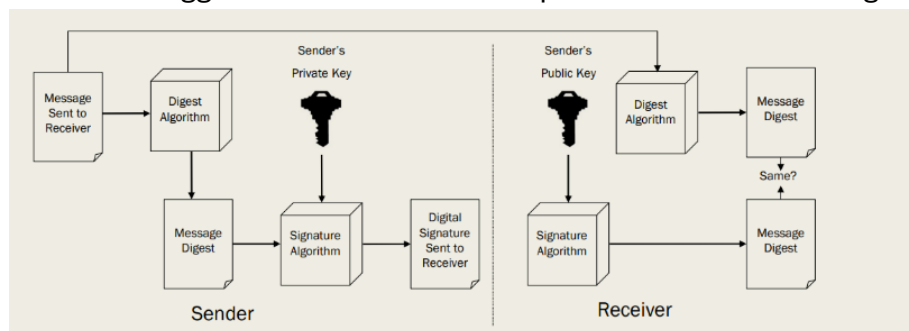


2.6 Firme digitali

Una **firma digitale** è un'**impronta digitale hash** di un dato che accompagna il dato stesso, fornendo una prova che il messaggio **proviene** effettivamente dal **mittente** legittimo.

Il messaggio viene prima trasformato in un **valore hash**. Questo hash viene poi **firmato** con la **chiave privata** del **mittente**. Il messaggio e la firma digitale vengono inviati al destinatario.

Il destinatario riceve il messaggio e calcola il **valore hash** del dato ricevuto utilizzando lo stesso algoritmo di hashing. Utilizzando la **chiave pubblica** del mittente, decifra l'impronta ricevuta e verifica che sia uguale al valore hash calcolato. Se i digest corrispondono, significa che il messaggio non è stato alterato e proviene dal mittente legittimo.



Una firma digitale **non garantisce la confidenzialità dei dati**. Per garantire sia la **confidenzialità** che l'**integrità**, il **mittente** cifra il messaggio cifrato + firma con la **chiave pubblica del destinatario**, per garantirne la confidenzialità. Il **destinatario** decifra il

messaggio con la propria **chiave privata**. Viene poi utilizzato il processo spiegato precedentemente.

2.7 Decentralized Identifiers

Un **Decentralized Identifier (DID)** è un identificatore digitale progettato per identificare un soggetto senza dipendere necessariamente da un'autorità centrale che assegni e gestisca l'identificatore. Un DID può riferirsi a una persona, un'organizzazione, un dispositivo o un'altra entità ed è associato a meccanismi crittografici che permettono al suo possessore di dimostrarne il controllo. Il DID, tuttavia, **non certifica da solo l'identità reale o le caratteristiche del soggetto**.

Un DID ha generalmente la seguente struttura. La prima parte, **did**, è una stringa fissa che indica che si ha Decentralized Identifier. La seconda parte, il **DID method**, specifica invece come viene registrato il DID, per esempio attraverso una blockchain o il web. La terza parte è il valore identificativo univoco. Ad esempio un DID può essere "did:etheraum:123456789abcdef".

Quando un DID viene creato, di solito è associato a una **coppia di chiavi crittografiche**, una **privata** e una **pubblica**.

Un **DID Document** è la struttura dati associata a un **DID** che contiene le informazioni pubbliche necessarie per verificare crittograficamente chi controlla quell'identificatore e per sapere come può essere utilizzato.

Per verificare una credenziale associata a quel DID, un verifier deve prima effettuare la **DID Resolution**, cioè recuperare il DID Document corrispondente. All'interno del documento trova uno o più **verification methods**, che indicano quali **chiavi pubbliche** o altri materiali crittografici sono autorizzati a essere utilizzati per quel DID.

Il processo di verifica può essere descritto nel seguente modo. Il controller del DID possiede una **chiave privata**, che mantiene segreta, e utilizza tale chiave per firmare digitalmente un messaggio o una credenziale. Il verifier riceve il dato firmato, recupera il DID dell'autore della firma e risolve tale DID per ottenere il relativo DID Document. Dal documento individua quindi il **verification method** utilizzato e recupera la corrispondente **chiave pubblica**. Con questa chiave verifica la firma digitale ricevuta.

Se la firma risulta valida, il verifier può concludere che il dato è stato firmato da chi possiede la **chiave privata corrispondente alla chiave pubblica autorizzata nel DID Document** e che il contenuto firmato non è stato modificato dopo la firma.

Il documento può anche contenere dei **service endpoint**, cioè indirizzi di servizi associati al DID.

Esempio DID Document:

```
"verificationMethod": [  
  {  
    "id": "did:example:123#key-1",  
    "type": "Ed25519VerificationKey2020",  
    "controller": "did:example:123",  
    "publicKeyMultibase": "zExampleKeyValue"  
  }  
]
```


Per recuperare le informazioni associate a un DID viene utilizzato un processo chiamato **DID Resolution**. Un componente chiamato **DID Resolver** riceve un DID come input, identifica il relativo DID method e applica la procedura prevista da quel metodo per ottenere il corrispondente **DID Document** e i relativi metadata. Le modalità di recupero del documento dipendono infatti dal DID method.

Nel contesto delle **Verifiable Credentials**, i DID possono essere utilizzati per identificare l'issuer, l'holder o il credential subject. Ad esempio, quando un verifier riceve una credenziale firmata da un issuer identificato tramite DID, può risolvere il DID dell'issuer, ottenere il relativo DID Document e recuperare la chiave pubblica necessaria per verificare crittograficamente la credenziale. La corretta verifica della firma dimostra che la credenziale è stata prodotta utilizzando il materiale crittografico associato all'issuer, ma non implica automaticamente che tale issuer debba essere considerato affidabile.

3. Librerie utilizzate

3.1 OpenZeppelin Contracts e Governance

OpenZeppelin Contracts è una libreria di smart contract scritti in Solidity che mette a disposizione componenti modulari e riutilizzabili per implementare funzionalità comuni nelle applicazioni Ethereum, come token, sistemi di governance e meccanismi di controllo degli accessi.

L'utilizzo di questi componenti permette di sviluppare nuovi smart contract partendo da implementazioni già esistenti, evitando di riscrivere da zero funzionalità comuni.

Questo approccio si basa principalmente sull'**inheritance** di Solidity: un nuovo contratto può ereditare uno o più moduli OpenZeppelin, riutilizzandone le funzionalità e, quando necessario, **estendendo o modificando il comportamento di alcune funzioni tramite override**.

Per implementare un sistema di governance, **OpenZeppelin** utilizza un'architettura **modulare**, composta da un contratto principale, chiamato **Governor**, e da una serie di estensioni che aggiungono funzionalità specifiche.

Il **Governor** rappresenta il componente centrale del sistema e gestisce il ciclo di vita di una proposta: la sua creazione, l'apertura della votazione, la registrazione dei voti, la determinazione del risultato e l'eventuale esecuzione della decisione approvata.

Le diverse regole della governance vengono poi definite attraverso moduli aggiuntivi.

GovernorVotes stabilisce da quale sorgente viene ricavato il **potere di voto** dei partecipanti, ad esempio da un token **ERC20Votes**. **GovernorCountingSimple** definisce come vengono espressi e conteggiati i voti, distinguendo tra voti favorevoli, contrari e astensioni.

GovernorSettings permette di configurare aspetti come il tempo che deve trascorrere prima dell'inizio di una votazione, la sua durata e il potere di voto minimo necessario per creare una proposta. **GovernorVotesQuorumFraction** stabilisce invece la quantità minima di partecipazione necessaria affinché una votazione sia considerata valida, calcolandola come una frazione del voting power complessivo.

L'esecuzione delle proposte può infine essere affidata a un **TimelockController**, collegato al Governor tramite **GovernorTimelockControl**. In questo caso, una proposta approvata non

viene eseguita immediatamente, ma viene prima messa in coda e può essere applicata solo dopo il trascorrere di un intervallo di tempo prestabilito.

3.1.1 Governor

Il contratto **Governor** fornisce le primitive necessarie per gestire il ciclo di vita delle **proposal**. Una proposta viene creata tramite la funzione **propose()**.

Ogni **proposal** può contenere una o più operazioni che dovranno essere eseguite in caso di approvazione. Per ciascuna operazione vengono specificati in input un **target**, cioè l'indirizzo del contratto destinatario, un eventuale valore in **ETH** da inviare a tale contratto e la **calldata** che rappresenta la funzione da eseguire con i relativi parametri. Alla proposta è inoltre associata una descrizione testuale.

Quando viene chiamata la funzione **propose()**, il **Governor** crea una nuova proposta contenente le azioni da eseguire precedentemente specificate.

Il **Governor** determina inoltre il **timepoint di snapshot**, cioè il momento di riferimento in cui viene fissato il potere di voto dei partecipanti e calcolato il quorum della proposta. L'inizio della votazione avviene dopo il periodo definito dal **votingDelay**, mentre il **votingPeriod** stabilisce per quanto tempo la proposta rimane aperta al voto. La **deadline** indica quindi il momento in cui la votazione termina.

La creazione della proposta genera inoltre l'evento **ProposalCreated** e produce un identificativo univoco, chiamato **proposalId**, utilizzato per identificare la proposta durante tutte le successive fasi del processo di governance.

Il Governor offre una funzione **state(uint256 proposalId)** che restituisce lo **stato** corrente di una **proposta**, identificata tramite il relativo **proposalId**. Una proposta può trovarsi in diversi stati, definiti dall'enum **ProposalState**, tra cui **Pending**, quando la proposta è stata creata, ma la votazione non è ancora iniziata; **Active**, quando la votazione è attualmente in corso, **Succeeded**, quando la votazione è terminata e la proposta ha soddisfatto le condizioni necessarie per essere approvata; **Defeated**, quando la proposta non ha raggiunto il quorum oppure non ha ottenuto i voti necessari per essere approvata; **Queued**, quando la proposta approvata è stata inserita nella coda per la successiva esecuzione; **Executed**, quando le operazioni previste dalla proposta sono state eseguite; **Canceled**, quando la proposta è stata annullata prima della sua esecuzione.

La funzione **proposalSnapshot(uint256 proposalId)** restituisce il **timepoint di riferimento della proposta**, cioè il momento utilizzato dal sistema di governance per determinare il **potere di voto dei partecipanti**. Infatti, il voting power di un partecipante non viene calcolato al momento del voto, ma al momento del **proposal snapshot**. In questo modo, un partecipante non può acquisire successivamente allo snapshot voting power per influenzare le votazioni.

La funzione **proposalDeadline(uint256 proposalId)** restituisce il **momento di chiusura del periodo di votazione** per la proposta.

La funzione **castVote(uint256 proposalId, uint8 choice)** permette a un partecipante di **esprimere il proprio voto su una proposta attiva**, prendendo in input l'ID della proposta, e la scelta di voto, che può assumere i valori **Against**, **For** oppure **Abstain**. Quando viene chiamata, il Governor verifica che la proposta sia nella fase di votazione e che quell'account **non abbia già votato** sulla stessa proposta. Poi recupera il **voting power dell'account al timepoint di snapshot della proposta**, e passa il voting power e la scelta effettuata al

meccanismo di conteggio. Il voto viene quindi registrato e il relativo peso viene aggiunto al conteggio della scelta effettuata.

3.1.2 ERC20Votes e GovernorVotes

OpenZeppelin mette a disposizione il modulo **ERC20Votes**, un'estensione di ERC-20 progettata per supportare i sistemi in cui il potere di voto dei membri di una governance deriva da un governance token.

A differenza di un normale token ERC-20, che permette principalmente di conoscere il balance corrente di un account, ERC20Votes mantiene informazioni sul **voting power nel tempo**, rendendo possibile recuperare il potere di voto associato a un determinato momento passato.

Il possesso di token rappresenta le voting units disponibili, mentre il potere di voto effettivo viene attribuito attraverso un meccanismo di delegation, rappresentato dalla funzione **delegate(address delegatee)**. Un account può delegare il proprio voting power a un altro indirizzo oppure delegarlo a se stesso attraverso una **self-delegation**, in modo da partecipare direttamente alla governance oppure attribuire il proprio potere di voto a un altro membro.

Per conservare lo **storico del voting power**, ERC20Votes utilizza dei **checkpoint**. Ogni volta che il potere di voto di un account cambia, ad esempio dopo una delegazione o una variazione del numero di token che contribuiscono al voto, viene registrato un nuovo checkpoint contenente il **nuovo voting power** e il relativo **timepoint**. È quindi possibile conoscere sia il potere di voto corrente, tramite **getVotes(address account)**, sia quello posseduto in un momento precedente tramite **getPastVotes(address account, uint256 blockNumber)**. Analogamente, **getPastTotalSupply(uint256 blockNumber)** permette di recuperare la quantità totale di voting units disponibile per tutti i membri ad un determinato timepoint.

Nel contesto di uso tipico, i membri di una DAO **ricevono token ERC20Votes** proporzionalmente allo stake che depositano. **Delegano poi i propri voti**, affinché i checkpoints relativi al voting power vengano creati. Quando una proposta entra nella fase di voto, il contratto di governance utilizza **getPastVotes()** per recuperare il **voting power posseduto da ciascun account al timepoint di snapshot della proposta**, evitando che eventuali trasferimenti di token successivi influenzino il peso del voto.

Le funzioni descritte sono definite dall'interfaccia **IVotes**, che stabilisce un insieme comune di operazioni che un contratto di governance può utilizzare per ottenere informazioni sui voti.

Il modulo **GovernorVotes** collega il contratto **Governor** a un contratto che implementa l'interfaccia **IVotes**, come **ERC20Votes**, permettendo al sistema di governance di recuperare il voting power dei partecipanti durante una votazione.

Il **costruttore** di GovernorVotes riceve come parametro il contratto che implementa **IVotes** che deve essere utilizzato come **sorgente del voting power**. GovernorVotes conserva quindi il riferimento a questo contratto, permettendo al Governor di recuperare da esso il potere di voto dei partecipanti.

La funzione **getVotes(address account, uint256 timepoint)** restituisce il **voting power posseduto da un determinato account in uno specifico timepoint**, leggendo i dati dal token che implementa IVotes, chiamando internamente la funzione **getPastVotes()**. Nel caso in cui GovernorVotes sia collegato a un token **ERC20Votes**, il voting power viene ricavato dai checkpoint storici mantenuti dal token.

3.1.3 GovernorSettings, GovernorVotesQuorumFraction e GovernorVotesSuperQuorumFraction

GovernorSettings è un'estensione di Governor utilizzata per configurare alcuni parametri fondamentali del processo di governance, tra cui il **votingDelay**, cioè **l'intervallo di tempo** che deve trascorrere tra la **creazione** di una proposta e **l'inizio** della **votazione**, il **votingPeriod**, cioè la **durata** del periodo durante il quale i partecipanti **possono votare**, e il **proposalThreshold**, cioè il **potere di voto minimo** che un account deve possedere per poter **creare** una nuova **proposta**.

GovernorSettings permette inoltre di modificare tali valori attraverso il processo di governance stesso, la DAO può aggiornare la configurazione del proprio sistema di voto senza dover sostituire il contratto Governor.

Per la gestione del quorum viene fornito il modulo **GovernorVotesQuorumFraction**, che definisce il quorum come una frazione della total supply delle voting units relativa a uno specifico timepoint. Una proposta per essere valida deve raggiungere almeno il quorum dei voti. Se, ad esempio, il numeratore del quorum è pari a 10 e il denominatore è 100, il numero minimo di voti richiesto corrisponde al 10% della total supply considerata al momento dello snapshot. La funzione **quorum(uint256 timepoint)** calcola il **numero minimo di voti richiesti per raggiungere il quorum** rispetto ad un certo timepoint.

OpenZeppelin mette inoltre a disposizione **GovernorVotesSuperQuorumFraction**, un'estensione che introduce il **superquorum**, cioè una soglia più elevata di voti favorevoli che può consentire l'approvazione anticipata di una proposta. Il super-quorum viene calcolato come una frazione della total supply delle voting units relativa a uno specifico timepoint. A differenza del quorum ordinario, per il raggiungimento del superquorum vengono conteggiati **solamente i voti For**. Se durante il periodo di votazione i voti favorevoli raggiungono questa soglia e la proposta presenta anche una **maggioranza di voti favorevoli**, essa può passare allo stato **Succeeded** prima della normale **deadline**, senza attendere la conclusione del **votingPeriod**.

3.1.4 TimelockController e GovernorTimelockControl

L'**esecuzione** delle decisioni può controllata tramite **TimelockController**, uno smart contract che impone un **ritardo di tempo minimo** tra la *schedulazione di un'operazione*, derivante da una proposta di governance approvata e la **sua effettiva esecuzione sulla blockchain**.

Tale ritardo ha l'obiettivo di evitare che una decisione di governance produca immediatamente i propri effetti, lasciando ai partecipanti il tempo di osservare l'operazione programmata e reagire prima della sua applicazione.

Il **TimelockController** offre il seguente funzionamento. Un'operazione viene inizialmente **schedulata** e associata a un determinato timestamp. Finché non è trascorso il delay minimo rimane in attesa, mentre una volta trascorso tale intervallo diventa eseguibile. Il contratto utilizza inoltre un sistema di controllo degli accessi basato su ruoli. Il **PROPOSER_ROLE** permette di schedulare le operazioni, l'**EXECUTOR_ROLE** consente di eseguirle quando diventano disponibili e il **CANCELLER_ROLE** permette di cancellare operazioni pendenti. La funzione **schedule()** inserisce in coda un'operazione da eseguire **dopo un certo delay minimo**, la funzione **execute()** esegue **un'operazione già programmata e pronta** per essere eseguita una volta trascorsi il delay, mentre la funzione **cancel(id)** rimuove dalla coda un'operazione id **prima che venga eseguita**.

Il collegamento tra Governor e TimelockController viene realizzato attraverso **GovernorTimelockControl**, che impone ad una proposta che supera con successo la votazione di non essere eseguita direttamente dal Governor, ma di essere inserita nella coda del TimelockController, in modo da essere eseguita solo dopo che il ritardo previsto è trascorso.

La funzione **queue()** prende una proposta **approvata dal voto** e la **invia al timelock per essere schedulata**. La funzione **execute()** prende una proposta che è stata schedulata nel timelock e, dopo che il delay è trascorso, **chiede** al timelock di **eseguire** le azioni programmate. La funzione **cancel()** **cancella** una **proposta** precedentemente schedulata nel timelock, se non è ancora stata eseguita.

3.2 Veramo

Veramo è un framework modulare utilizzato per sviluppare applicazioni basate su **Verifiable Data** e **Self-Sovereign Identity (SSI)**. Fornisce strumenti per creare e gestire **Decentralized Identifiers (DIDs)**, chiavi crittografiche, **Verifiable Credentials (VCs)** e **Verifiable Presentations (VPs)**, mettendo a disposizione i principali componenti necessari per implementare i **ruoli** di **issuer**, **holder** e **verifier**.

Il componente centrale del framework è il **Veramo Agent**, un oggetto configurabile che coordina le funzionalità dell'applicazione. L'Agent viene costruito attraverso un'architettura basata su **plugin**, ognuno dei quali aggiunge una specifica funzionalità. In questo modo è possibile configurare l'Agent includendo solamente i componenti necessari per il sistema che si vuole realizzare.

Tra i principali componenti vi è il **Key Manager**, responsabile della gestione delle **chiavi crittografiche** e delle operazioni di firma necessarie agli altri moduli. Il **DID Manager** permette invece di creare e gestire i **Decentralized Identifiers**, utilizzando differenti DID provider. Il **DID Resolver** ha il compito di risolvere un DID e recuperare le informazioni associate, come il relativo **DID Document** e i metodi di verifica necessari per identificare le chiavi pubbliche associate al soggetto.

Per la gestione delle credenziali, Veramo mette a disposizione il **Credential Plugin**, che fornisce le operazioni necessarie per creare e verificare **Verifiable Credentials** e **Verifiable Presentations**. Un issuer può utilizzare il Veramo Agent per creare una VC contenente i **claim** relativi al credential subject e proteggerla crittograficamente utilizzando le chiavi gestite dall'Agent. La credenziale può successivamente essere presentata a un verifier, che può utilizzare Veramo per verificarne la provenienza, l'integrità e la validità crittografica.

3.3 Hardhat

Hardhat è un ambiente di sviluppo utilizzato per creare, testare e distribuire smart contract Ethereum.

Una delle sue funzionalità principali è la **compilazione** degli smart contract. Il codice scritto in Solidity viene compilato in bytecode, che può poi essere eseguito dalla Ethereum Virtual Machine. Hardhat permette inoltre di eseguire **test automatici**, utili per verificare che le funzioni dei contratti producano il comportamento atteso prima del deployment su una rete reale.

Hardhat mette a disposizione anche una **rete Ethereum locale simulata**, che permette di eseguire smart contract e transazioni in un ambiente controllato. In questo modo è possibile provare il funzionamento del sistema senza utilizzare una blockchain pubblica e senza spendere ETH reali. Durante i test locali, Hardhat fornisce anche informazioni utili per il **debugging**, come messaggi di errore relativi alle transazioni fallite.

Hardhat supporta inoltre il **deployment** degli smart contract, cioè la loro pubblicazione su una rete Ethereum. Per questa fase può essere utilizzato **Hardhat Ignition**, che permette di definire e gestire in modo strutturato il processo di deployment dei diversi contratti.

4. Proposal

4.1 Competence-Based Governance Model

L'approccio classico usato nelle DAO, in cui il voting power di un membro è proporzionale ai soldi depositati, presenta un limite importante: possedere più token non significa necessariamente avere maggiori competenze rispetto all'argomento su cui si sta votando. Un membro incompetente ma che ha versato moltissimi soldi può influenzare le decisioni, votando e facendo passare proposte totalmente contro l'interesse dell'organizzazione.

L'obiettivo di questa proposta di ricerca è costruire un sistema in cui il potere di voto è basato sia sui soldi depositati dal membro, sia dalle sue competenze. Se un membro è molto competente in un determinato ambito, ottiene un maggiore voting power.

La governance definisce nel proprio smart contract i gradi di competenza che un membro può avere.

Per calcolare il voting power di un membro al momento del voto si utilizza una formula pesata basata sulla quantità di stake depositato, su quanto pesa la parte relativa allo stake nel voting power totale, sul grado di competenza del membro, e su quanto pesa nella formula la parte di competenze.

La parte dedicata allo **stake** della formula viene calcolata al momento del **minting di token** via deposito di ETH da parte del membro. La parte relativa alle **competenze** viene calcolata nel momento in cui il membro **presenta** una Verifiable Credential che attesta le proprie competenze.

4.1.1 Verifica delle competenze di un membro tramite VC

Applicando il modello SSI, si ha un **issuer** che fornisce una Verifiable Credential ad un membro **holder**, firmando digitalmente il suo contenuto con la propria chiave privata. Quando il membro vuole effettuare un upgrade di competenza all'interno della DAO, per incrementare il proprio voting power, presenta la credenziale alla DAO, che svolge in questo contesto il ruolo di **verifier on-chain**.

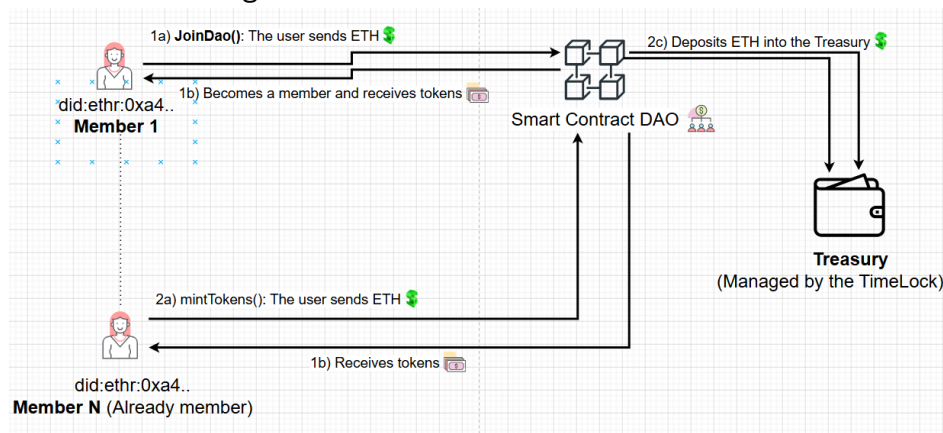
La DAO mantiene internamente una **lista** degli **issuer** che considera **fidati**. All'interno di una Verifiable Credential è presente la firma dell'**issuer**. Il **verifier** effettua l'operazione inversa. Ricrea il digest della VC utilizzando le stesse operazioni usate dall'issuer per effettuare la firma. Controlla che il digest finale combaci bit-a-bit con quello presente nella VC, in modo da verificare che la VC non sia stata alterata.

A partire dal digest e dalla firma, viene recuperato l'address relativo alla chiave pubblica che ha prodotto la firma. Il chiamante confronta poi questo address con la lista dei trusted issuer. Se l'indirizzo è presente nella lista, la VC è verificata.

4.1.2 Modello di Governance

In questa architettura, chiunque può unirsi alla DAO, inviando ETH e ricevendo in cambio token di voto, senza che l'inclusione di un nuovo membro o il mining di nuovi token debba passare da una proposal.

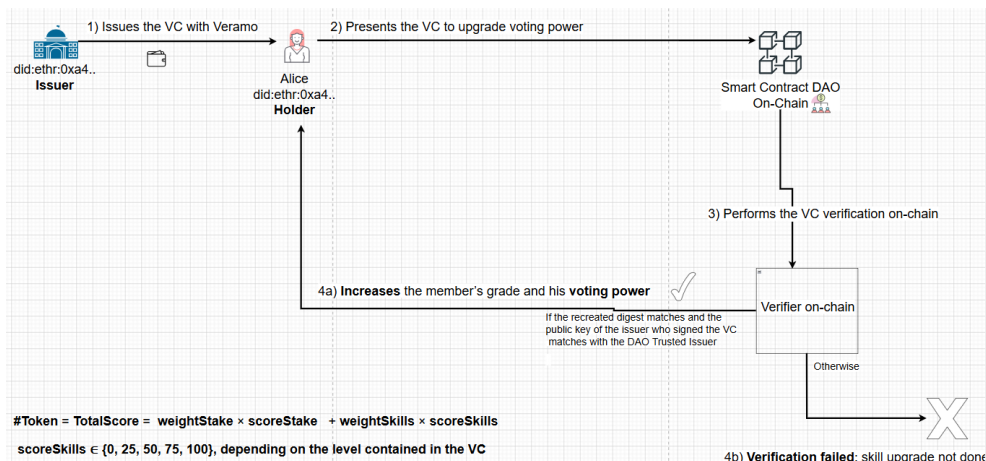
Lo stake depositato dai membri nella DAO viene inviato ad un contratto **Treasury**, a cui può accedere solo la governance tramite il TimeLockController.



Quando l'utente entra nella DAO, parte dal livello di competenza base.

Il potere di voto di un membro della DAO dipende dalla **quantità di ETH** depositati nella DAO e dal **livello di competenza** del membro.

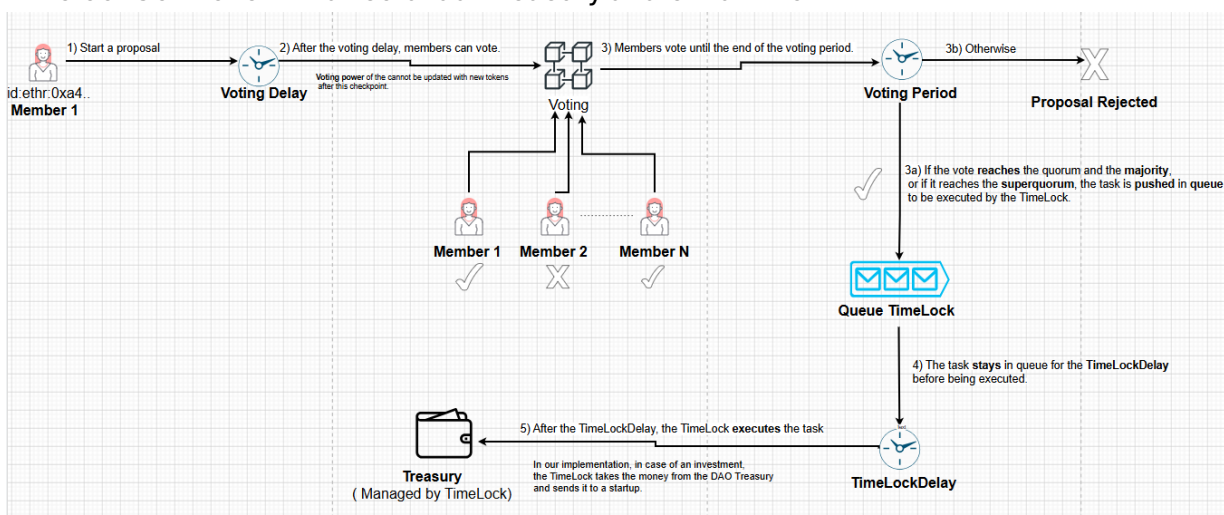
Se in futuro un membro fornisce una **VC** che prova il suo livello di competenza, la DAO verifica on-chain se la VC è valida ed è firmata da un **Issuer** considerato come **fidato** dalla DAO. Se la verifica passa, il membro ottiene maggiore potere di voto. Se il membro deposita in futuro altri ETH, il proprio voting power aumenta secondo la formula prestabilita. L'upgrade del livello di competenza può essere eseguito da un membro senza passare dal processo di governance.



Il processo di governance inizia con la **creazione di una proposta** da parte di un membro della DAO. Le **proposte** vengono utilizzate per **decidere collettivamente** come **investire i soldi** depositati nel **treasury**. Solo il **TimeLockController** è autorizzato ad **eseguire** le **azioni** approvate relative alle **proposal**, e a **investire i soldi** del treasury.

Qualunque membro può creare una proposta, senza limiti relativi a stake depositato o competenze possedute. Dopo un periodo di **voting delay** dalla creazione, la proposta entra nella fase di voto e viene fissato lo **snapshot** utilizzato per determinare il voting power dei partecipanti. Infatti grazie all'uso degli snapshot è **impossibile acquistare** potere di voto durante la votazione di una proposal, poiché anche se i token vengono mintati, la proposal tiene conto della **totalSupply** di **token** del **blocco** in cui parte il **votingPeriod**.

Durante il **voting period**, i membri esprimono il proprio voto fino alla scadenza prevista. Se la proposta raggiunge il **quorum** e la maggioranza richiesta, oppure soddisfa le condizioni di **superquorum**, viene approvata e inserita nella coda del **TimeLockController**, mentre in caso contrario viene respinta. Una volta approvata, l'operazione rimane in attesa per il **TimeLock Delay** e può essere eseguita solo al termine di tale intervallo. Nella nostra implementazione, nel caso di investimenti di denaro da parte della DAO verso un indirizzo esterno, il TimeLockController invia i soldi dal Treasury a tale indirizzo.



Sia nel caso di mint per unione alla DAO, che in caso di mint successivo, i soldi depositati dai memb vengono **inviati** dallo smart contract del token direttamente a quello del **treasury** della DAO.

In questa implementazione la DAO utilizza un **quorum** del 20%, per cui per essere **valida** una proposta devono votare **almeno il 20%** della **totalSupply**, e un **superquorum** del 70%, per

cui la proposta viene **approvata immediatamente** se viene raggiunto il **70%** di voti favorevoli, senza **attendere** la fine del **voting period**.

4.1.3 Criticità sull'utilizzo di token per rappresentare il voting power composto

Nel caso in cui si utilizza un unico token per rappresentare sia il voting power derivato da stake, sia il voting power derivato da competenze, si ha il problema che i token derivati da **stake** possono essere **trasferibili**, mentre i token derivati da **skill** non **dovrebbero** essere **trasferibili**.

Per risolvere questo problema, si rende il token composto solo dalla parte **stake**. Il voting power viene rappresentato non dal solo **token**, ma dalla composizione tra il token, che rappresenta la parte stake, più la parte skill, presa esternamente da uno **stato** salvato internamente nel contratto.

Per fare questo, si mantiene nello smart contract dei **checkpoint** aggiuntivi che salvano ad ogni upgrade di competenze la parte **skill** del voting power di un utente e la **totalSupply** della parte **skill** di tutti gli utenti. Il voting power di un utente viene quindi derivato dalla somma pesata dei token posseduti da tale utente più il grado di skill mantenuto nel checkpoint. La totalSupply dipende non solo dalla totalSupply dei token relativi allo stake, ma dalla somma tra la totalSupply derivata da token e da skill.

4.1.4 Calcolo del voting power in DAO basate su topic

La proposta di tesi vuole progettare una DAO in cui le skill di un utente forniscono un diverso voting power in base al **topic** della votazione.

Ad esempio, in una DAO universitaria si possono avere più topic di governance (Computer Science, Electronics, Finance, Health), e per ogni diverso topic la stessa skill vale diversamente.

Sia la soluzione che utilizza un token unico per rappresentare il voting power, sia quella che prevede la composizione di token e checkpoint, non sono valide per implementare una governance basata su topic.

Per implementare tale soluzione, la parte relativa allo stake di un utente viene implementata da un token. Per la parte **skill**, **per ogni topic** sono necessari due **checkpoint**: uno che tiene traccia **per ogni utente** del **valore** della **skill** per tale topic, ed uno che tiene traccia della somma del voting power derivato da skill di tutti gli utenti per tale topic.

4.1.5 Formula per calcolare il voting power di un membro

Per calcolare il voting power di un membro della DAO in un dato timepoint viene utilizzata la seguente formula:

VotingPower(account, topic, timepoint) = **VotingPowerStake**(account, timepoint) + **VotingPowerSkill**(account, topic, timepoint)

La componente economica riprende la formula:

VotingPowerStake(account, timepoint) = **weightStake** × **scoreStake**(account, timepoint),
scoreStake(account, timepoint) = **min**(stakeDeposited / MAX_DEPOSIT, 1) × 100 ∈ [0, 100].

Viene impostato un limite massimo di stake depositabile. Lo score derivato da stake assume dominio da 0 a 100. Se viene depositato più del massimo depositabile, lo score assume

valore 100, altrimenti assume come valore il rapporto tra lo stake depositato e il massimo depositabile, moltiplicato per 100. Lo score ottenuto viene moltiplicato per un coefficiente che pesa quanto vale il voting power derivato da stake in percentuale sul voting power totale.

La componente relativa alle skill usa la formula:

VotingPowerSkill(account, topic, timepoint) = **weightSkill** × **scoreSkills**

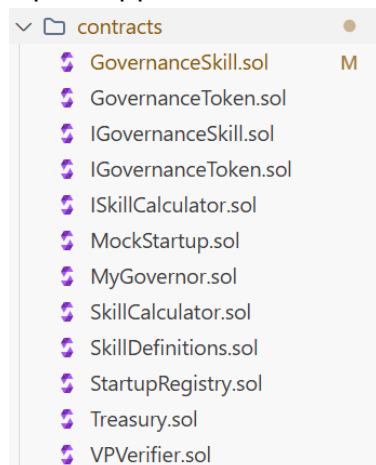
Lo score viene calcolato in base al valore della skill posseduta dal membro rispetto al topic della votazione. Per ottenere il voting power derivato da skill, lo score viene poi moltiplicato al coefficiente di peso **weightSkill** dato dalla DAO.

La somma tra **weightSkill** e **weightStake** deve essere uguale al 100%. Ad esempio, si può avere 100% stake / 0% skill, 75% / 25%, 50% / 50%, 25% / 75%.

4.2 Implementazione

4.2.1 Struttura del progetto

Il progetto utilizza Solidity, HardHat, Typescript, Veramo e le API offerte dalla libreria OpenZeppelin.



Il contratto principale di **governance** che gestisce l'intero ciclo di vita delle proposte, dalla creazione di una proposta, alla votazione fino all'esecuzione è **MyGovernor.sol**, che eredita dalla libreria OpenZeppelin Governor e da altre sue estensioni.

MyGovernor.sol calcola il voting power di ciascun membro sommando il proprio voting power derivato da stake e da skill, mantenuti rispettivamente dal contratto **GovernanceToken.sol** e dal contratto **GovernanceSkill.sol**.

Le interfacce **IGovernanceToken.sol** e **IGovernanceSkill.sol** definiscono le interfacce esposte esternamente dai moduli che implementano la gestione del voting power.

GovernanceToken.sol implementa il token ERC20Votes che funge da componente economica di voto, mentre **GovernanceSkill.sol** gestisce la componente di competenza, mantenuta in checkpoint storici, che vengono aggiornati quando un utente presenta una nuova VC che certifica le sue competenze.

La validazione crittografica di queste credenziali è affidata alla libreria **VPVerifier.sol**, che verifica on-chain, secondo lo standard EIP-712, le firme delle VC presentate dagli utenti, assicurandosi che siano state emesse da un issuer attendibile.

La logica di traduzione delle competenze in punteggi numerici è invece separata nel contratto **SkillCalculator.sol**, che implementa l'interfaccia **ISkillCalculator.sol**. L'uso di un'interfaccia permette di sostituire la formula di calcolo in futuro senza modificare il resto del sistema.

Le definizioni condivise di topic e competenze riconosciute dalla DAO sono centralizzate nella libreria **SkillDefinitions.sol**, accessibile dagli altri contratti senza costi di storage aggiuntivi.

La gestione dei fondi è affidata al contratto **Treasury.sol**, una cassaforte che può erogare ETH verso indirizzi esterni esclusivamente su ordine del sistema di governance, garantendo che nessun singolo membro possa spostare i fondi autonomamente.

Gli indirizzi eleggibili a ricevere investimenti sono mantenuti nel contratto **StartupRegistry.sol**, un registro on-chain modificabile solo tramite decisione collettiva.

Il contratto **MockStartup.sol** fornisce un'implementazione fittizia utilizzata per simulare il comportamento di una startup destinataria di fondi.

4.2.2 GovernanceToken.sol

Lo Smart Contract **GovernanceToken.sol** implementa il token ERC20Votes usato per votare nella DAO e implementa le funzionalità necessarie per gestire la componente stake del voting power.

Chiunque può unirsi alla DAO chiamando il metodo **joinDAO()** e depositando ETH, ottenendo proporzionalmente dei token di governance. I membri possono acquistare token aggiuntivi chiamando **mintTokens()**. I token ottenuti vengono calcolati in base alla formula della componente stake del voting power spiegata precedentemente.

Il token eredita ERC20 per le funzionalità base del token (transfer, balanceOf, ecc.), e ERC20Votes per la gestione del potere di voto nella DAO, con checkpoint basati sul blocco di inizio votazione e delega del potere di voto.

Vengono usate come **costanti** all'interno del contratto il **deposito massimo per membro**, definito arbitrariamente come 100 ETH e il denominatore **basis points** che rappresenta il 100% per effettuare i calcoli in %, definito come 10 000.

```
// Costanti DAO

uint256 public constant MAX_DEPOSIT = 100 ether;    //Deposito massimo per membro: 100 E
uint256 public constant BASIS_POINTS = 10_000;     /// Denominatore basis points per ef

// Stato configurabile

uint256 public immutable weightStake;    // Peso della componente stake nella formula di
uint256 public immutable weightSkill;    // Peso della componente skill.

// Timelock e deployer non cambiano mai dopo il deploy.
// Il deployer serve solo per il bootstrap iniziale; poi le modifiche passano dal Timelock
address public immutable timelock;       /// Indirizzo del TimelockController, cioè l'esec
address public immutable deployer;       /// Indirizzo del deployer, cioè la il token, us

address public treasury;                 // Indirizzo del Treasury a cui vengono inoltrati gli ETH
```

Come **variabili di stato** vengono usati l'**indirizzo del TimelockController**, immutabile, usato per effettuare operazioni di governance; l'**indirizzo del deployer**, anch'esso immutabile, usato per chiamare setTreasury() al deploy della DAO; l'**indirizzo del Treasury**, usato per inviare gli ETH ricevuti dai joinDAO() e dai mintTokens().

Si ha il **peso** della **componente economica** weightStake, immutabile, in percentuale basis points, e il **peso** della **componente accademica** weightSkill, anche esso in percentuale basis points.

Si hanno due mapping: Una mappa **stakeDeposited** che tiene traccia di quanti ETH ha depositato ogni membro, e una mappa **isMember** che tiene traccia degli indirizzi membri della DAO.

```
mapping(address => uint256) public stakeDeposited;
mapping(address => bool) public isMember;      //
```

Lo smart contract utilizza due decorator. Si ha decorator **onlyTimelock** che obbliga la funzione interna ad essere eseguita solo dal **TimeLockController**, e un decorator **onlyDeployer()** che obbliga la funzione interna ad essere eseguita solo dal **deployer**.

```
modifier onlyTimelock() {
    if (msg.sender != timelock) revert OnlyTimelock();
    _;
}

/// Decorator che obbliga la funzione interna ad essere
modifier onlyDeployer() {
    if (msg.sender != deployer) revert OnlyDeployer();
    _;
}
```

Il **costruttore** dello smart contract prende come input l'indirizzo del TimelockController della DAO, il peso della componente skill e della componente stake per il calcolo del voting power. Inizializza il token, passando staticamente nome e abbreviativo del token, controlla che il timelock non abbia indirizzo 0 e che la somma tra le due percentuali corrisponda a 10 000. Poi imposta l'indirizzo del timelock, quello del deployer all'indirizzo msg.sender del chiamante, e i pesi weightStake e weightSkill ai valori passati in input.

```
constructor(
    address _timelock,
    uint256 _weightSkill,
    uint256 _weightStake
) ERC20("CompetenceDAO Token", "COMP") ERC20Permit("CompetenceDAO Token") {
    if (_timelock == address(0)) revert ZeroAddress();
    if (_weightSkill + _weightStake != BASIS_POINTS) revert InvalidWeights();

    // Pesì e timelock sono configurati al deploy. I pesì potranno poi cambiar
    timelock = _timelock;
    deployer = msg.sender;
    weightStake = _weightStake;
    weightSkill = _weightSkill;
}
```

La funzione **setTreasury()** è una funzione di setup one shot che **imposta l'indirizzo del Treasury**. Prende in input l'indirizzo del treasury e può essere chiamata una sola volta, solo dal deployer. È necessaria perché il Treasury viene deployato dopo il GovernanceToken.

```
function setTreasury(address _treasury) external onlyDeployer {
    if (treasury != address(0)) revert TreasuryAlreadySet();
    if (_treasury == address(0)) revert ZeroAddress();
    treasury = _treasury;
}
```

La funzione **calculateStakeTokens()** calcola i token da mintare per la componente stake della formula di calcolo del voting power, ossia:

VotingPowerStake(account, timepoint) = **weightStake** × **scoreStake**(account, timepoint),

scoreStake(account, timepoint) = **min**(stakeDeposited / MAX_DEPOSIT, 1) × 100 ∈ [0, 100].

Il metodo calcola **weightStake** × Δ **scoreStake**, ottenendo il numero di token che deve essere incrementato.

La funzione prende il vecchio score, il nuovo score, effettua la differenza e moltiplica per **weightStake**, calcolando i token da mintare. Il risultato è in wei ($\times 10^{18}$).

Ad esempio, se si ha **weightStake**=5000 bp, **previousDeposit**=5 ETH, **depositAmount**=3 ETH, la funzione calcolerà **oldScore**=5, **newScore**=8, Δ **scoreStake**=3, **token** = $(3 \times 5000 \times 10^{18}) / 10000 = 1.5 \times 10^{18}$ token

```
function _calculateStakeTokens(uint256 depositAmount, uint256 previousDeposit) internal view returns (uint256) {
    uint256 oldEffective = previousDeposit > MAX_DEPOSIT ? MAX_DEPOSIT : previousDeposit;
    uint256 newTotal = previousDeposit + depositAmount;
    uint256 newEffective = newTotal > MAX_DEPOSIT ? MAX_DEPOSIT : newTotal;

    uint256 effectiveDiff = newEffective - oldEffective;

    // Calcolo ottimizzato per non perdere i decimali di precisione.
    // Equivale alla logica originale (scoreDiff * weightStake * 1e18) / BASIS_POINTS
    return (effectiveDiff * 100 * weightStake * 1e18) / (MAX_DEPOSIT * BASIS_POINTS);
}
```

La funzione **joinDAO()** è una funzione usata dagli utenti per entrare nella DAO, chiamabile da chiunque, senza dover passare da una proposal. Può essere chiamata solo dagli utenti che non sono ancora membri della DAO. La funzione controlla che il treasury abbia un indirizzo assegnato, che il deposito sia superiore a 0 e inferiore al deposito massimo consentito. Calcola il numero di token da ricevere in base al deposito effettuato, utilizzando le funzioni di utility implementate precedentemente.

Imposta il nuovo membro come attivo e registra il deposito effettuato. I token vengono mintati e inviati al membro. La funzione trasferisce gli ETH ricevuti dal membro al treasury.

Se un utente ha depositato il massimo depositabile, non può più effettuare minting.

```

function joinDAO() external payable {
    if (treasury == address(0)) revert TreasuryNotSet();
    if (isMember[msg.sender]) revert AlreadyMember();
    if (msg.value == 0) revert ZeroDeposit();
    if (msg.value > MAX_DEPOSIT) revert ExceedsMaxDeposit();
    if (stakeDeposited[msg.sender] >= MAX_DEPOSIT) revert MaxDepositReached();

    uint256 tokenAmount = _calculateStakeTokens(msg.value, 0);
    if (tokenAmount == 0) revert DepositTooSmall();

    isMember[msg.sender] = true;
    stakeDeposited[msg.sender] = msg.value;

    _mint(msg.sender, tokenAmount);
    (bool ok, ) = treasury.call{value: msg.value}("");
    if (!ok) revert TreasuryTransferFailed();
    emit MemberJoined(msg.sender, msg.value, tokenAmount);
}

```

La funzione **increaseStake()** è utilizzata per permettere ad un membro di mintare i token successivamente all'ingresso nella DAO, depositando ETH. La funzione controlla che il membro sia effettivamente un membro della DAO, che il treasury abbia un indirizzo assegnato e che il deposito inviato sia superiore a 0. Controlla che gli ETH depositati dal membro sommati a quelli che sta per depositare non superino MAX_DEPOSIT. Calcola il numero di token da ricevere in base al deposito effettuato. Viene aggiornato il conto degli ETH depositati dall'utente. Vengono mintati i token. Gli ETH vengono trasferiti direttamente al Treasury.

```

function increaseStake() external payable {
    if (!isMember[msg.sender]) revert NotMember();
    if (msg.value == 0) revert ZeroDeposit();
    if (treasury == address(0)) revert TreasuryNotSet();
    if (stakeDeposited[msg.sender] >= MAX_DEPOSIT) revert MaxDepositReached();
    if (stakeDeposited[msg.sender] + msg.value > MAX_DEPOSIT) revert ExceedsMaxDeposit();

    uint256 newTokens = _calculateStakeTokens(msg.value, stakeDeposited[msg.sender]);
    if (newTokens == 0) revert DepositTooSmall();

    stakeDeposited[msg.sender] += msg.value;
    _mint(msg.sender, newTokens);
    (bool ok, ) = treasury.call{value: msg.value}("");
    if (!ok) revert TreasuryTransferFailed();
    emit StakeIncreased(msg.sender, msg.value, newTokens);
}

```

4.2.3 GovernanceSkill.sol

GovernanceSkill.sol è lo smart contract che gestisce la componente skill per il calcolo del voting power.

Per aumentare il proprio potere di voto, un membro può effettuare un **upgrade di competenza** presentando una **VC**. Se la verifica va a buon fine, viene chiamato il metodo **upgradeCompetence()** per aumentare il grado di competenza del membro chiamante per ogni topic di votazione esistente. Per calcolare lo score derivato dalle skills possedute da un utente, per ogni topic, viene chiamato il contratto SkillCalculator.sol.

Vengono usate come **costanti** il denominatore **basis points** che rappresenta il 100% per effettuare i calcoli in %, definito come 10 000,

Vengono usate come **costanti** il **deposito massimo per membro** (1 il denominatore basis points (10 000) che rappresenta il 100% per effettuare i calcoli in %, e il **Domain separator EIP-712 universale**, usato nella verifica VC.

Il **Domain separator EIP-712** universale viene usato come costante poiché precalcolato a compile-time, in modo che il valore viene incorporato direttamente nel bytecode senza occupare storage e senza costi di SLOAD.

```
bytes32 public constant UNIVERSAL_DOMAIN_SEPARATOR =
    keccak256(
        abi.encode(
            keccak256("EIP712Domain(string name,string version)"),
            keccak256(bytes("Universal VC Protocol")),
            keccak256(bytes("1"))
        )
    );
```

Come **variabili di stato** vengono usati l'**indirizzo del TimelockController**, usato per effettuare operazioni di governance; l'**indirizzo del deployer**; l'**indirizzo del Trusted Issuer**, usato per il setup iniziale; il **peso** della **componente** di skill nella formula del voting power; il riferimento immutabile all'interfaccia che astrae il token di governance; il riferimento immutabile all'interfaccia che astrae lo skill calculator.

```
uint256 public immutable weightSkill;    // Peso d

// Timelock e deployer non cambiano mai dopo il dep
// Il deployer serve solo per il bootstrap iniziale
address public immutable timelock;        /// Indiriz
address public immutable deployer;        /// Indiriz

IGovernanceToken public immutable governanceToken;
ISkillCalculator public immutable skillCalculator;
```

Si ha un mapping **trustedIssuers**, che mantiene gli indirizzi degli Issuer considerati **attendibili** dal contratto, che firmano le Verifiable Credential contenenti le skill dei membri, accompagnato da un **counter** che conta il numero dei trusted issuer; una mappa **memberDID** che associa ogni membro al suo DID; una mappa **didToAddress** che associa ogni DID al suo indirizzo, per garantire che lo stesso DID non venga registrato da due address; un mapping **memberSkills** che contiene per ogni indirizzo la lista degli hash delle skill del membro, e un mapping che per ogni indirizzo, per ogni skill, contiene un booleano che dice se il membro ha tale skill.

```
mapping(address => bool) public trustedIssuers; // Issuer attendibi
uint256 public trustedIssuerCount;

mapping(address => bytes32) public memberDID;    // Mappa che associ
mapping(bytes32 => address) public didToAddress; // Garantisce che 1

// Storage canonico delle skill di un membro. Le skill sono salvate
// per evitare stringhe nello storage e confrontarle in modo economi
mapping(address => bytes32[]) public memberSkills;
mapping(address => mapping(bytes32 => bool)) public memberHasSkill;
```

Il contratto mantiene un checkpoint rappresentante il voting power derivato da skill di un utente per un determinato topic, in un dato timepoint, implementato come un mapping che associa l'indirizzo di un utente, a un intero che rappresenta un topic, ad un checkpoint Trace208 che salva storicamente un intero legato ad un timepoint.

Si ha un altro checkpoint che rappresenta la somma cumulativa di tutti i voting power derivati da skill di tutti gli utenti, per un certo topic, ad un certo timepoint.

```
// Checkpoint skill per topic

// _skillVotesCheckpoints[account][topicId]
// storico cumulativo del voting power derivato da skill dell'utente, per quel determinato topic.
// Associa l'indirizzo di un membro, al mapping che va dal topicId al voting power relativo.
mapping(address => mapping(uint256 => Checkpoints.Trace208)) private _skillVotesCheckpoints;

// _totalSkillSupplyCheckpoints[topicId]:
// Tiene traccia della somma cumulativa di tutti i VP-skill emessi per quel topic, per tutti gli utenti.
// Associa il topicId al voting power da skill totale accumulato.
mapping(uint256 => Checkpoints.Trace208) private _totalSkillSupplyCheckpoints;
```

Lo smart contract utilizza due decorator. Si ha decorator **onlyTimelock** che obbliga la funzione interna ad essere eseguita solo dal **TimeLockController**, e un decorator **onlyDeployer()** che obbliga la funzione interna ad essere eseguita solo dal **deployer**.

```
modifier onlyTimelock() {
    if (msg.sender != timelock) revert OnlyTimelock();
    _;
}

/// Decorator che obbliga la funzione interna ad essere
modifier onlyDeployer() {
    if (msg.sender != deployer) revert OnlyDeployer();
    _;
}
```

Il **costruttore** prende come input l'indirizzo del TimelockController, del token di governace e dello skill calculator, e il peso della componente skill weightSkill per il calcolo del voting power. Inizializza il token di governance, il timelock, il deployer al msg.sender, il peso weightSkill e lo skill calculator ai parametri in input ricevuti.

```
constructor(
    address _governanceToken,
    address _timelock,
    uint256 _weightSkill,
    address _skillCalculator
) {
    if (_governanceToken == address(0)) revert ZeroAddress();
    if (_timelock == address(0)) revert ZeroAddress();
    if (_weightSkill > BASIS_POINTS) revert InvalidWeights();
    governanceToken = IGovernanceToken(_governanceToken);
    timelock = _timelock;
    deployer = msg.sender;
    weightSkill = _weightSkill;
    skillCalculator = _validateSkillCalculator(_skillCalculator);
    emit SkillCalculatorSet(_skillCalculator);
}
```

La funzione **registerDID** registra il DID di un membro. Un membro può registrare un solo DID e lo stesso DID non può essere usato da due address. Verifica che il msg.sender sia un membro

della DAO e che non abbia già registrato un DID. Effettua l'hash del DID, controlla che non sia già stato registrato. In tal caso, registra nei mapping le associazioni address -> DID e DID -> address.

```
function registerDID(string calldata _did) external {
    if (!governanceToken.isMember(msg.sender)) revert NotMember();
    if (bytes(_did).length == 0) revert EmptyDID();
    if (memberDID[msg.sender] != bytes32(0)) revert DIDAlreadyRegistered();

    bytes32 didHash = keccak256(bytes(_did));
    if (didToAddress[didHash] != address(0)) revert DIDAlreadyBound();

    memberDID[msg.sender] = didHash;
    didToAddress[didHash] = msg.sender;

    emit DIDRegistered(msg.sender, didHash);
}
```

La funzione **setTrustedIssuer** configura gli indirizzi degli issuer che la DAO considera fidati, che possono firmare le VC che attestano le competenze dei membri. Il primo issuer può impostarlo solo il deployer. Dopo la prima configurazione, solo la governance può cambiarlo o aggiungere altri issuer.

```
function setTrustedIssuer(address _issuer) external {
    if (_issuer == address(0)) revert ZeroAddress();
    if (trustedIssuers[_issuer]) revert TrustedIssuerAlreadySet();

    if (trustedIssuerCount == 0) {
        if (msg.sender != deployer) revert OnlyDeployer();
    } else {
        if (msg.sender != timelock) revert OnlyTimeLock();
    }

    trustedIssuers[_issuer] = true;
    trustedIssuerCount++;
    emit TrustedIssuerAdded(_issuer);
}
```

La funzione **removeTrustedIssuer** rimuove un issuer dall'insieme fidato. Può farlo solo la governance, tramite il TimeLockController.

Le funzioni di supporto **_mergeSkills()** e **_addValidSkill**, dato l'indirizzo di un membro e una lista di hash identificativi di skill, aggiunge le skill valide secondo la libreria SkillDefinitions, che non sono già possedute da un utente, nei mapping che rappresentano rispettivamente le skill possedute da un utente e l'appartenenza di una skill ad un utente.

```
function _mergeSkills(address member, bytes32[] memory skillIds) internal returns (bytes32[] memory) {
    bytes32[] storage skills = memberSkills[member];
    uint256 added;
    uint256 skillCount = skillIds.length;
    for (uint256 i = 0; i < skillCount; i++) {
        if (_addValidSkill(member, skills, skillIds[i])) {
            ++added;
        }
    }
    if (added > 0) emit MemberSkillsMerged(member, added, skills.length);
    return skills;
}

function _addValidSkill(address member, bytes32[] storage skills, bytes32 skillId) private returns (bool) {
    if (!SkillDefinitions.isValidSkill(skillId)) revert InvalidSkill(skillId);
    if (memberHasSkill[member][skillId]) return false;

    memberHasSkill[member][skillId] = true;
    skills.push(skillId);
    return true;
}
```

La funzione **upgradeCompetenceWithVP()** esegue l'upgrade delle skills di un membro tramite VC. Usa la libreria **VPVerifier.sol** per verificare una VC firmata e applica l'upgrade se la VC è valida. La funzione controlla se il membro è esistente e se è configurato nella DAO almeno un issuer fidato. Controlla se il DID del membro è coerente con il DID nel credentialSubject. Il typehash del dominio EIP-712 è precalcolato come constant, utilizzando 0 gas di hashing.

La funzione recupera l'address del firmatario usando le funzioni della libreria VPVerifier sulla firma EIP-712 contenuta nella VC. Controlla se l'issuer recuperato è uguale ad uno dei trustedIssuer. In caso positivo, chiama il metodo **_mergeSkills()** per aggiungere le skill verificate al membro.

Costruisce una stringa proof sintetica persistita on-chain.

Esegue l'aggiornamento del voting power derivato da skill del membro chiamando direttamente **_performUpgrade**, senza passare dalla governance.

```
function upgradeSkillWithVC(
    VPVerifier.VerifiableCredential memory _vc,
    bytes memory _issuerSignature
) external {
    if (!governanceToken.isMember(msg.sender)) revert NotMember();
    if (trustedIssuerCount == 0) revert TrustedIssuerNotSet();
    bytes32 didHash = memberDID[msg.sender];
    if (didHash == bytes32(0)) revert NoDIDRegistered();
    if (keccak256(bytes(_vc.credentialSubject.id)) != didHash) revert DIDMismatch();

    address recovered = VPVerifier.recoverIssuer(_vc, _issuerSignature, UNIVERSAL_DOMAIN_SEPARATOR);
    if (!trustedIssuers[recovered]) revert UntrustedIssuer();

    bytes32[] memory mergedSkills = _mergeSkills(msg.sender, _skillIds(_vc.credentialSubject.skills));

    bytes32 issuerDidHash = keccak256(bytes(_vc.issuer.id));
    bytes32 proofHash = _credentialSubjectProofHash(_vc.credentialSubject);
    _performUpgrade(msg.sender, mergedSkills, proofHash);

    emit SkillUpgradedWithVC(msg.sender, issuerDidHash);
}
```

Il metodo **_credentialSubjectProofHash()** crea una proof sintetica di un upgrade di competenza andato a buon fine, hashando il typehash con gli elementi del Credential Subject della VC, creando una stringa sintetica da persistere on chain.

```
function _credentialSubjectProofHash(
    VPVerifier.CredentialSubject memory credentialSubject
) private pure returns (bytes32) {
    return keccak256(
        abi.encode(
            VC_PROOF_TYPEHASH,
            VPVerifier.hashCredentialSubject(credentialSubject)
        )
    );
}
```

La funzione **_performUpgrade()** si occupa di effettuare nella DAO le modifiche al voting power del membro dopo che la VC è stata verificata.

Si ha che la formula per calcolare il voting power relativo alla componente skill, per ogni topic t, sia: **VotingPowerSkill(skills[], topic) = skillsScoreForTopic(skills[], t) × weightSkill × 10¹⁸ / BASIS_POINTS**.

Per calcolare skillsScoreForTopic(skills[], t) si usa esternamente l'interfaccia ISkillCalculator. La funzione prende il blocco attuale, prende la lista di topic, e per ogni topic, chiama lo SkillCalculator per calcolare il voting power dell'utente per tale topic. Si chiama poi la funzione che aggiorna i checkpoint.

```
function _performUpgrade(
    address _member,
    bytes32[] memory _skills,
    bytes32 _proofHash
) internal {
    uint48 blk = clock();
    uint256[] memory scores = skillCalculator.calculateAllVP(_skills);
    if (scores.length != SkillDefinitions.TOPIC_COUNT) revert InvalidCalculator();

    for (uint256 topicId = 0; topicId < SkillDefinitions.TOPIC_COUNT; topicId++) {
        uint256 score = scores[topicId];
        if (score > 100) revert InvalidCalculator();
        uint256 newVP = (score * weightSkill * 1e18) / BASIS_POINTS;

        _writeSkillVotes(_member, topicId, newVP, blk);
    }

    emit SkillUpgraded(_member, _proofHash);
}

function _writeSkillVotes(
    address account,
    uint256 topicId,
    uint256 newVP,
    uint48 checkpointKey
) internal {
    Checkpoints.Trace208 storage userTrace = _skillVotesCheckpoints[account][topicId];
    uint208 oldVotes = userTrace.latest();
    uint208 newVotes = SafeCast.toUint208(newVP);
    if (newVotes == oldVotes) return;

    Checkpoints.Trace208 storage totalTrace = _totalSkillSupplyCheckpoints[topicId];
    uint208 oldTotal = totalTrace.latest();

    userTrace.push(checkpointKey, newVotes);

    if (newVotes > oldVotes) {
        totalTrace.push(checkpointKey, oldTotal + (newVotes - oldVotes));
    } else {
        totalTrace.push(checkpointKey, oldTotal - (oldVotes - newVotes));
    }
}
```

La funzione **_writeSkillVotes()** aggiunge, dopo un upgrade di skill, il voting power derivato da skill aggiuntivo per un dato topic per un dato membro ai checkpoint relativi ad un membro e alla total supply di ogni topic.

La funzione crea un record sul checkpoint relativo al voting power del membro per un dato topic, salvando il nuovo valore, in coppia al relativo timepoint.

Poi crea il record per il checkpoint sulla totalSupply. Confronta il voting power attuale del membro con quello dell'ultimo checkpoint. Se il voting power del membro è aumentato, aggiunge la differenza, mentre se è diminuito sottrae la differenza.

4.2.4 IGovernanceToken.sol e IGovernanceSkill.sol

I moduli IGovernanceToken.sol e IGovernanceSkill.sol sono interfacce che espongono i metodi da implementare per costruire una governance basata su un voting power composto da una componente economica e da una componente basata sulle competenze.

I metodi esposti sono i metodi minimi di cui hanno bisogno il contratto Governor.sol, i rispettivi GovernanceToken o GovernanceSkill, e un applicativo utente per implementare una governance di questo tipo.

Il modulo IGovernanceToken.sol espone i metodi necessari per verificare se un membro fa parte della DAO, utile a GovernanceToken, per unirsi alla DAO, per mintare nuovi token e per ottenere il quantitativo di token di un membro.

```
interface IGovernanceToken is IERC5805 {

    /// @dev Necessaria al modulo skill per consentire upgrade solo ai membri.
    function isMember(address member) external view returns (bool);

    /// @dev Flusso utente minimo per entrare nella DAO e aumentare lo stake.
    function joinDAO() external payable;
    function increaseStake() external payable;
    function getStakeScore(address _member) external view returns (uint256);
}
```

Il modulo IGovernanceToken.sol espone i metodi necessari per verificare se un topic è supportato dalla DAO, registrare il DID di un membro, conoscere le skill di un membro, effettuare l'upgrade di skill mostrando una VC, e ottenere i valore dei checkpoint, relativi ad un utente o all'intera total supply, per un dato topic.

```
interface IGovernanceSkill {

    /// @dev Il Governor usa questa funzione per accettare solo proposte con topic supportati.
    function isValidTopic(uint256 topicId) external view returns (bool);

    /// @dev Flusso utente minimo: registrare DID, aggiornare le skill via VC e leggerle.
    function registerDID(string calldata _did) external;
    function getMemberSkills(address member) external view returns (bytes32[] memory);
    function upgradeSkillWithVC(
        VPVerifier.VerifiableCredential memory _vc,
        bytes memory _issuerSignature
    ) external;

    /// @dev Letture necessarie al Governor per VP, quorum e snapshot topic-aware.
    function getSkillVotes(address account, uint256 topicId) external view returns (uint256);
    function getPastSkillVotes(address account, uint256 topicId, uint256 timepoint) external view returns (uint256);
    function getTotalSkillSupply(uint256 topicId) external view returns (uint256);
    function getPastTotalSkillSupply(uint256 topicId, uint256 timepoint) external view returns (uint256);
}
```

4.2.5 SkillDefinitions.sol

SkillDefinitions è una **libreria** che raccoglie le definizioni condivise dei **topic** e delle **skill** supportati dalla DAO. Il modulo assegna un identificativo numerico ai topic e rappresenta ciascuna skill tramite un identificatore bytes32 ottenuto con keccak256. Fornisce inoltre funzioni per convertire una stringa nello skillId corrispondente, verificare se un topic o una skill sono supportati dalla DAO e recuperare l'elenco completo delle skill e dei topic supportati.

In questa implementazione, i topic di votazione supportati sono WEB3, AI, HEALTH, e ENTERPRISE, mentre le skill che un membro può avere sono SMART CONTRACT, MACHINE LEARNING, TOKENOMICS, DIGITAL HEALTH, DATA ANALYSIS e BACKEND JAVA.

```
library SkillDefinitions {
    uint256 internal constant TOPIC_WEB3 = 0;
    uint256 internal constant TOPIC_AI = 1;
    uint256 internal constant TOPIC_HEALTH = 2;
    uint256 internal constant TOPIC_ENTERPRISE = 3;
    uint256 internal constant TOPIC_COUNT = 4;

    bytes32 internal constant SKILL_SMART_CONTRACTS = keccak256(bytes("smart-contracts"));
    bytes32 internal constant SKILL_MACHINE_LEARNING = keccak256(bytes("machine-learning"));
    bytes32 internal constant SKILL_TOKENOMICS = keccak256(bytes("tokenomics"));
    bytes32 internal constant SKILL_DIGITAL_HEALTH = keccak256(bytes("digital-health"));
    bytes32 internal constant SKILL_DATA_ANALYSIS = keccak256(bytes("data-analysis"));
    bytes32 internal constant SKILL_BACKEND_JAVA = keccak256(bytes("backend-java"));
}
```

4.2.6 ISkillCalculator.sol e SkillCalculator.sol

ISkillCalculator fornisce l'interfaccia minima usata da GovernanceSkill per calcolare il voting power derivato dalle skill. Espone solo l'operazione batch calculateAllVP necessaria per calcolare in base alle skills di un utente, gli score di voting power per ogni topic.

L'utilizzo di un'interfaccia permette di **separare la logica di governance dalla logica utilizzata per calcolare il voting power derivato dalle competenze**. In questo modo, GovernanceSkill non dipende da una specifica implementazione di SkillCalculator, ma richiede solamente che venga rispettata l'interfaccia ISkillCalculator.

Chi riutilizza i moduli del progetto può quindi sviluppare un proprio calcolatore delle competenze, definendo **formule, pesi e criteri di valutazione differenti per ciascun topic**, senza dover modificare il contratto di governance. È sufficiente che il nuovo contratto implementi la funzione calculateAllVP() prevista dall'interfaccia.

```
interface ISkillCalculator {
    /// @notice Restituisce gli score (0-100) per tutti i topic supportati in una sola chiamata.
    function calculateAllVP(bytes32[] calldata skills) external view returns (uint256[] memory);
}
```

SkillCalculator è il contratto che implementa la logica di **calcolo del voting power derivato dalle skill** dei membri della DAO. Implementa l'interfaccia ISkillCalculator e, tramite la funzione calculateAllVP(), riceve in input l'insieme delle skill possedute da un utente e restituisce uno **score compreso tra 0 e 100 per ciascun topic supportato**.

```
function calculateAllVP(bytes32[] calldata skills) external pure override returns (uint256[] memory scores) {
    scores = new uint256[](SkillDefinitions.TOPIC_COUNT);
    for (uint256 topicId = 0; topicId < SkillDefinitions.TOPIC_COUNT; topicId++) {
        scores[topicId] = _calculateVP(topicId, skills);
    }
}
```

Il valore di ogni skill dipende dal topic considerato. Ad esempio, la skill SMART CONTRACTS contribuisce maggiormente al topic WEB3, mentre la skill MACHINE LEARNING ha un peso maggiore nel topic AI. Il contratto può inoltre applicare dei **boost aggiuntivi** quando un utente possiede particolari combinazioni di skill rilevanti per uno specifico topic. Il punteggio finale viene infine limitato a un massimo di 100.

```
function _scoreBackendJava(uint256 topicId) private pure returns (uint256) {
    if (topicId == SkillDefinitions.TOPIC_WEB3) return 5;
    if (topicId == SkillDefinitions.TOPIC_AI) return 10;
    if (topicId == SkillDefinitions.TOPIC_HEALTH) return 5;
    return 40;
}
```

```

function _boost(
    uint256 topicId,
    bool hasSmartContracts,
    bool hasMachineLearning,
    bool hasTokenomics,
    bool hasDigitalHealth,
    bool hasDataAnalysis,
    bool hasBackendJava
) private pure returns (uint256) {
    if (topicId == SkillDefinitions.TOPIC_WEB3 && hasSmartContracts && hasTokenomics) {
        return 20;
    }

    if (topicId == SkillDefinitions.TOPIC_AI && hasMachineLearning && hasDataAnalysis) {
        return 20;
    }

    if (topicId == SkillDefinitions.TOPIC_HEALTH && hasDigitalHealth && hasDataAnalysis) {
        return 20;
    }

    if (topicId == SkillDefinitions.TOPIC_ENTERPRISE && hasBackendJava && hasDataAnalysis) {
        return 15;
    }

    return 0;
}

```

4.2.7 Governor.sol

MyGovernor è il contratto principale che gestisce il processo di governance della DAO, coordinando l'intero ciclo di vita delle proposte: dalla loro creazione, alla fase di votazione, fino all'eventuale esecuzione tramite TimelockController.

Il contratto eredita e combina diversi moduli di governance di OpenZeppelin, riutilizzando funzionalità già disponibili per la gestione delle proposal (Governor.sol), il conteggio dei voti (GovernorCountingSimple.sol), i parametri di governance (GovernorSettings.sol), il quorum (GovernorVotesQuorumFraction.sol), il superquorum (GovernorVotesSuperQuorumFraction.sol), il voting power storico (GovernorVotes.sol) e il timelock (GovernorTimelockControl).

La principale estensione introdotta da MyGovernor riguarda la gestione di una governance **basata su topic e competenze**. Ogni proposta viene associata a un **topicId**, che identifica l'ambito a cui appartiene. Il voting power di ciascun membro viene quindi calcolato combinando due componenti: il **voting power derivato dallo stake**, ottenuto dal token di governance, e il **voting power derivato dalle skill**, recuperato da GovernanceSkill in relazione al topic della proposta.

Entrambe le componenti vengono lette rispetto al **timepoint di snapshot**, così da impedire che variazioni successive dello stake o delle competenze modifichino il peso di voto già determinato per quella proposta.

Anche il calcolo di **quorum e super-quorum** viene adattato al modello multi-topic. Le soglie non vengono infatti applicate soltanto alla supply storica del token, come succede se non vengono applicate modifiche, ma alla **supply totale votabile del topic**, ottenuta sommando la total supply della componente di stake e la componente di skill relativa a tale topic.

Come stato interno si ha due riferimenti immutabili a IGovernanceToken e IGovernanceSkill, e un mapping che associa ogni proposalId al suo topicId, usato quando si vuole recuperare il topicId di una proposta, in modo da recuperare la supply del voting power legato alle skill del topic collegato alla proposta.

```

IGovernanceToken public immutable governanceToken;
IGovernanceSkill public immutable governanceSkill;
mapping(uint256 => uint256) public proposalTopic;

```

Il costruttore riceve in input l'IGovernanceToken, l'IGovernanceSkill, il TimelockController, il numero di blocchi di attesa prima dell'inizio del voto, la durata della finestra di voto, la soglia minima di voti per poter creare una proposta, il quorum in percentuale della supply totale

votabile al blocco di snapshot della proposta, e il superquorum. I contratti ereditati vengono inizializzati con tali parametri.

```
constructor(  
    IGovernanceToken token_,  
    IGovernanceSkill skill_,  
    TimelockController timelock_,  
    uint48 votingDelay_,  
    uint32 votingPeriod_,  
    uint256 proposalThreshold_,  
    uint256 quorumNumerator_,  
    uint256 superQuorumNumerator_  
)  
  
    Governor("MyGovernor")  
    GovernorSettings(votingDelay_, votingPeriod_, proposalThreshold_)  
    GovernorVotes(token_)  
    GovernorVotesQuorumFraction(quorumNumerator_)  
    GovernorVotesSuperQuorumFraction(superQuorumNumerator_)  
    GovernorTimelockControl(timelock_)  
{  
    governanceToken = token_;  
    governanceSkill = skill_;  
}
```

La funzione **proposeWithTopic()** crea una proposta associandola a un topicId. Sostituisce la funzione **propose()**, aggiungendo il supporto al parametro **topicId** non presente nell'implementazione originale, poichè ogni proposta deve dichiarare l'ambito di competenza tramite topicId, su cui viene poi calcolato il voting power derivato da skill, e a catena il quorum e superquorum. La funzione verifica che il topicId sia valido, chiama super.propose() per creare la proposta e poi salva il topicId in proposalTopic[proposalId].

```
function proposeWithTopic(  
    address[] memory targets,  
    uint256[] memory values,  
    bytes[] memory calldatas,  
    string memory description,  
    uint256 topicId  
) external returns (uint256 proposalId) {  
    _validateTopicId(topicId);  
    proposalId = super.propose(targets, values, calldatas, description);  
    proposalTopic[proposalId] = topicId;  
    emit ProposalTopicSet(proposalId, topicId);  
}
```

La funzione **_getVotes()** durante una votazione recupera e compone il voting power di un membro, derivato da stake e da skill.

La formula per comporre il voting power di un membro, per un determinato topic, è:

VotingPowerTotal(account, timepoint, topic) = token.getPastVotes(account, timepoint) + governanceSkill.getPastSkillVotes(account, topic, timepoint)

La funzione prende in input l'account del membro, il blocco di snapshot della proposta e i parametri della proposta, in cui è contenuto il topicId.

Si prendono i voti di stake usando la funzione getVotes() di ERC20Votes. Se i parametri sono vuoti, quindi non si ha topicId, la funzione ritorna solo i voti provenienti da stake. Altrimenti si decodifica il topicId dai parametri della proposta, si verifica che sia valido, e si recupera dal

checkpoint che salva le skill di un membro, il voting power derivato da skill del membro nel topic della proposta, in quel tale timepoint. Si ritorna la somma tra voting power derivato da stake e da skill.

```
function _getVotes(
    address account,
    uint256 timepoint,
    bytes memory params
) internal view override(Governor, GovernorVotes) returns (uint256) {
    if (!governanceToken.isMember(account)) {
        return 0;
    }

    uint256 stakeVotes = super._getVotes(account, timepoint, params);

    if (params.length == 0) {
        return stakeVotes;
    }
    if (params.length != 32) revert InvalidVoteParams();

    uint256 topicId = abi.decode(params, (uint256));
    _validateTopicId(topicId);
    uint256 skillVotes = governanceSkill.getPastSkillVotes(
        account,
        topicId,
        timepoint
    );

    return stakeVotes + skillVotes;
}
```

Senza modifiche, il contratto Governor prende come total supply votabile solamente la total supply relativa al token ERC20Votes, senza tenere conto della componente skill. Questo invalida i meccanismi di votazione, quorum e superquorum.

La funzione **_totalVotingPowerForTopic** calcola la total supply votabile per un dato topic, tenendo conto sia della componente stake che della componente skill: **supplyTopic = supplyStake + supplySkill(topicId)**. Per recuperare le total supply vengono usati i checkpoint gestiti da ERC20Votes e governanceSkill.

```
function _totalVotingPowerForTopic(
    uint256 topicId,
    uint256 timepoint
) internal view returns (uint256) {
    uint256 stakeSupply = governanceToken.getPastTotalSupply(timepoint); //supply storica del token
    uint256 skillSupply = governanceSkill.getPastTotalSkillSupply(
        topicId,
        timepoint
    ); //supply skill storica del topic
    return stakeSupply + skillSupply;
}
```

La funzione **_quorumForTopic()** calcola il quorum di una votazione relativa ad un determinato topic, ad un certo timepoint. Recupera la total supply tramite la funzione precedente e applica la percentuale di quorum. Senza questa modifica, il quorum sarebbe stato calcolato solo in base alla total supply derivata da stake, invalidando la votazione. In modo analogo viene effettuata la modifica per passare la total supply alla funzione che calcola il superquorum.


```

function _quorumForTopic(
    uint256 topicId,
    uint256 timepoint
) internal view returns (uint256) {
    return
        Math.mulDiv(
            _totalVotingPowerForTopic(topicId, timepoint),
            quorumNumerator(timepoint),
            quorumDenominator()
        );
}

```

Si effettua l'override della funzione **_quorumReached**, che decide se una proposta ha superato il quorum, sostituendo GovernorCountingSimple._quorumReached, che chiamerebbe quorum() al posto di _quorumForTopic(). La funzione recupera il timepoint di snapshot della proposta e il topicId dal relativo mapping. Calcola il quorum relativo al topic della proposta usando la funzione ad hoc precedente. Recupera il numero di voti che hanno votato FOR e Abstain per la proposta. Confronta questo numero con il quorum sulla total supply di voting power. Se supera q, la proposta è valida.

```

function _quorumReached(
    uint256 proposalId
) internal view override(Governor, GovernorCountingSimple) returns (bool) {
    uint256 snapshot = proposalSnapshot(proposalId);
    uint256 topicId = proposalTopic[proposalId];
    uint256 q = _quorumForTopic(topicId, snapshot);

    (, uint256 forVotes, uint256 abstainVotes) = proposalVotes(proposalId);
    return q <= forVotes + abstainVotes;
}

```

La funzione **_superQuorumReached** è stata rimossa in OpenZeppelin 5.4, quindi la logica topic-specifica del superquorum viene incorporata nella funzione state().

La funzione **state()** restituisce lo stato attuale di una proposta, ritornando stati come Pending, Active, Succeeded, Defeated, Queued, Executed, Canceled.

Per implementare la logica di superquorum, viene fatto il seguente adattamento: durante lo stato Active, il numero di voti attuali viene confrontato con il superquorum calcolato sulla total supply del topic della proposta. Lo stato attuale della proposta viene recuperato da .state() di Governor. Se lo stato è Active, si verifica se il superquorum è raggiunto usando la formula custom. Se i voti FOR sono maggiori del superquorum della totalsupply, e se sono maggiori dei voti against, la proposta passa allo stato Succeeded, usando .state() di GovernorSuperQuorumFraction. Altrimenti resta in Active.

```

    returns (ProposalState)
{
    ProposalState baseState = Governor.state(proposalId);

    if (baseState == ProposalState.Active) {
        (, uint256 forVotes, ) = proposalVotes(proposalId);
        uint256 topicId = proposalTopic[proposalId];
        uint256 sq = _superQuorumForTopic(
            topicId,
            proposalSnapshot(proposalId)
        );

        if (forVotes >= sq && !_voteSucceeded(proposalId)) {
            // Delegando a super.state, sfruttiamo GovernorTimeLockControl
            // per ottenere il corretto stato (Succeeded/Queued/Executed).
            // Siccome sq (Topic) >= stakeSQ (Token), OZ riconoscerà il superquorum.
            return super.state(proposalId);
        }
        // Blocca early execution prematura causata dallo stakeSQ inferiore
        return ProposalState.Active;
    }

    return super.state(proposalId);
}

```

In questa implementazione si ha un **quorum** del **20%** e un **superquorum** del **70%**. Tali impostazioni sono modificabili parametricamente. Il contratto esegue inoltre l'override richiesto da Solidity per le funzioni di `votingDelay`, `votingPeriod`, `proposalThreshold`, `clock`, e `_execute`.

4.2.8 Treasury.sol

Treasury.sol è un contratto che conserva i **fondi** della DAO e permette di investirli in altri indirizzi, in questa implementazione startup, **solo** se l'operazione è stata **approvata** dalla governance e passa attraverso il **TimelockController**. Infatti il Treasury non è controllabile dal deployer né da nessun altro account, ma l'unico indirizzo che può chiamare la funzione **invest()** è il **TimelockController**. Questo garantisce che nessun singolo individuo possa spostare i fondi senza il consenso della DAO.

Il flusso di movimento dei fondi è il seguente. I membri depositano ETH mintando token. Un membro propone un investimento tramite il Governor. La comunità vota. Se approvata, la proposta viene messa in coda nel Timelock. Dopo il delay, il Timelock esegue `Treasury.invest()`, inviando gli ETH all'indirizzo designato dalla proposta.

Come variabili di stato abbiamo l'address del **TimelockController**, che è l'unico che può ordinare investimenti, un mapping che mantiene lo storico di tutti gli investimenti effettuati, con chiave l'indirizzo startup e valore gli ETH investiti su di essa.

Abbiamo un decorator usato per indicare che solo il Timelock può chiamare la funzione decorata. Il costruttore dello smart contract inizializza il Treasury prendendo in input e impostando l'indirizzo del TimelockController.

Si ha una funzione **deposit()** che permette a chiunque di depositare ETH nel Treasury. Questa viene chiamata dal GovernanceToken per depositare i soldi ricevuti al momento del mint dei token da parte dei membri.

```
// Decorator che indica che solo il Timelock può chiamare la funzione decorata
modifier onlyTimelock() {
    if (msg.sender != timelock) revert OnlyTimelock();
    _;
}

// Costruttore che inizializza il Treasury impostando l'indirizzo del TimelockController
constructor(address _timelock) {
    if (_timelock == address(0)) revert ZeroAddress();
    timelock = _timelock;
}

/// Funzione che permette a chiunque di depositare ETH nel Treasury
function deposit() external payable {
    emit Deposited(msg.sender, msg.value);
}
```

La funzione principale di Treasury è **invest()**, chiamabile solo dal TimeLock, che permette alla DAO di investire ETH in un indirizzo startup. Prende in input l'indirizzo della startup destinataria e l'importo in wei da investire. Viene chiamata dal Timelock dopo che una proposta di investimento è stata approvata e il delay è trascorso. Incrementa l'importo investito nella startup e trasferisce ETH alla startup.

```

function invest(
    address _startup,
    uint256 _amount
) external onlyTimeLock nonReentrant {
    // Controlli di sicurezza
    if (_startup == address(0)) revert ZeroAddress();
    if (address(this).balance < _amount) revert InsufficientBalance();

    // Registra l'investimento nello storico (Effects prima di Interactions)
    investedIn[_startup] += _amount;

    // Trasferisci ETH alla startup (Interactions)
    (bool success, ) = _startup.call{value: _amount}("");
    if (!success) revert TransferFailed();

    emit Invested(_startup, _amount);
}

```

Si ha anche una funzione **getBalance()** che restituisce il saldo attuale del Treasury.

4.2.9 StartupRegistry.sol

Contratto che mantiene un registro on-chain di indirizzi relativi a startup verso cui la DAO può investire. Invece di proporre investimenti verso indirizzi "random", la DAO può verificare che la startup sia registrata dai membri e attiva.

Solo la governance, quindi il TimelockController, può registrare o disattivare startup.

4.2.10 MockStartup.sol

MockStartup.sol è un contratto mock che simula una startup che riceve investimenti dalla DAO. Serve a verificare nei test il corretto funzionamento della logica di investimento, e che i fondi siano arrivati.

La funzione **receive()** permette al contratto di ricevere ETH e registra l'investimento.

La funzione **getBalance()** restituisce il saldo ETH attuale del contratto.

4.3 Implementazione della Verifica on-chain delle VC e Rilascio con Veramo

La DAO proposta permette di effettuare un upgrade del proprio voting power ad un membro nel caso in cui presenta una VC, rilasciata da un issuer fidato, contenente una lista di skill certificate. In base a tale skills, se la VC viene correttamente verificata, il voting power aumenta proporzionalmente. La verifica viene fatta on-chain, mediante uno smart contract, utilizzando lo standard EIP712.

4.3.1 Verifica on-chain delle VC: Libreria VPVerifier.sol

VPVerifier.sol è una libreria utilizzata per verificare on-chain una VC firmata utilizzando lo standard EIP-712. La libreria prende i dati della credenziale, ricostruisce esattamente lo stesso hash digest che era stato firmato off-chain, e poi utilizza la firma contenuta nella VC per recuperare la chiave pubblica di chi lo ha firmato.

Il contratto chiamante confronta poi l'indirizzo derivato dalla chiave pubblica con la propria lista di trusted issuer. Se coincide con uno degli indirizzi presenti nella lista, la VC è valida.

4.3.1.1 Costanti TypeHash

La libreria utilizza delle costanti per costruire il **typeHash**, ovvero l'hash della struttura dei campi che compongono la VC. I nomi e l'ordine dei campi devono essere identici a quelli usati off-chain durante l'emissione della VC. La libreria hasha la struttura del credentialSubject, con al suo interno i campi id, university, faculty, e skills.

```

bytes32 internal constant ISSUER_TYPEHASH = keccak256("Issuer(string id)");

bytes32 internal constant CREDENTIAL_SUBJECT_TYPEHASH =
    keccak256(
        "CredentialSubject("
        "string id,"
        "string university,"
        "string faculty,"
        "string[] skills"
        ")");

```

Ricostruisce poi il typeHash della struct principale VerifiableCredential, contenente issuer, issuanceDate e credentialSubject in modo annidato, come richiede EIP-712.

```

bytes32 internal constant VERIFIABLE_CREDENTIAL_TYPEHASH =
    keccak256(
        "VerifiableCredential("
        "Issuer issuer,"
        "string issuanceDate,"
        "CredentialSubject credentialSubject"
        ")");
    keccak256(
        "CredentialSubject("
        "string id,"
        "string university,"
        "string faculty,"
        "string[] skills"
        ")");
        "Issuer("
        "string id"
        ")");
    );

```

4.3.1.2 Struct che modellano i dati della VC

La libreria crea poi delle struct utili per contenere i dati della VC. Crea una struct contenente l'identità dell'issuer (DID).

Crea poi una struct contenente i dati informativi relativi all'holder, ovvero il CredentialSubject, i cui campi sono gli stessi già descritti.

Crea infine una struct contenente tutti i dati della VC informativi, da certificare. Comprende come campi le due struct precedenti, e la data di emissione.

```

/// Identità dell'issuer, espressa come DID.
struct Issuer {
    string id;
}

/// Struct contenente i dati certificati relativi all'holder.
struct CredentialSubject {
    string id;
    string university;
    string faculty;
    string[] skills; // Array dinamico di skill
}

/// Struct contenente la VC completa: issuer, data di emissione e credentialSubject.
struct VerifiableCredential {
    Issuer issuer;
    string issuanceDate;
    CredentialSubject credentialSubject;
}

```

4.3.1.3 Funzioni di hashing

La prima funzione si occupa di effettuare l'Hash EIP-712 della struct Issuer, hashando il typeHash e l'id dell'issuer.

```
// Hash EIP-712 della struct `Issuer`. Hasha il typeHash e l'id dell'issuer.
function hashIssuer(Issuer memory issuer) internal pure returns (bytes32) {
    return
        keccak256(abi.encode(ISSUER_TYPEHASH, keccak256(bytes(issuer.id))));
}
```

La seconda funzione si occupa di effettuare l'Hash EIP-712 della struct CredentialSubject, hashando il typeHash e tutti i valori assunti dai campi informativi dell'holder.

```
/// Hash EIP-712 della struct CredentialSubject con tutti i campi certificati.
function hashCredentialSubject(CredentialSubject memory cs) internal pure returns (bytes32) {
    return keccak256(
        abi.encode(
            CREDENTIAL_SUBJECT_TYPEHASH,
            keccak256(bytes(cs.id)),
            keccak256(bytes(cs.university)),
            keccak256(bytes(cs.faculty)),
            hashSkills(cs.skills)
        )
    );
}
```

La terza funzione si occupa di effettuare l'Hash EIP-712 della credenziale completa. Per campi struct annidati usa i rispettivi hash, chiamando le due funzioni precedenti sui dati completi.

```
// Hash EIP-712 della credenziale completa. Per campi struct annidati
// usa i rispettivi hash (`hashIssuer`, `hashCredentialSubject`), chiamando la funzione completa.
function hashVerifiableCredential(
    VerifiableCredential memory vc
) internal pure returns (bytes32) {
    return
        keccak256(
            abi.encode(
                VERIFIABLE_CREDENTIAL_TYPEHASH,
                hashIssuer(vc.issuer),
                keccak256(bytes(vc.issuanceDate)),
                hashCredentialSubject(vc.credentialSubject)
            )
        );
}
```

4.3.1.4 Recover Signer

La funzione di Recover Signer ricrea il digest EIP-712 usando le funzioni precedentemente implementate. Viene ricreato il digest EIP-712 concatenando il prefisso standard `0x1901`, il domainSeparator e il digest. Il digest finale deve combaciare bit-a-bit con quello usato dal firmatario off-chain. Se non corrisponde, i dati della VC sono stati alterati. A partire dal digest e dalla firma, viene recuperato l'address relativo alla chiave pubblica che ha prodotto la firma. Il chiamante confronta poi questo address con i trusted issuer. Se coincide con un trusted issuer, la VC è valida.

```
function recoverIssuer(
    VerifiableCredential memory vc,
    bytes memory signature,
    bytes32 domainSeparator
) internal pure returns (address signer) {
    bytes32 structHash = hashVerifiableCredential(vc);
    bytes32 digest = keccak256(
        | abi.encodePacked("\x19\x01", domainSeparator, structHash)
    );

    signer = ECDSA.recover(digest, signature);
}
```

4.3.1.2 Generazione e firma di VC con Veramo (Parte legacy da aggiornare)

Il modulo **Veramo** si occupa del rilascio di VC da parte di un Issuer che certifica delle skills ad una serie di Holder, in questo caso i membri della DAO.

Esempio VC

```
{
  "@context": [
    "https://www.w3.org/2018/credentials/v1"
  ],
  "type": [
    "VerifiableCredential",
    "UniversityDegreeCredential"
  ],
  "issuer": {
    "id": "did:ethr:sepolia:0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC"
  },
  "issuanceDate": "2026-03-24T18:35:43Z",
  "credentialSubject": {
    "id": "did:ethr:sepolia:0x23618e81E3f5cdF7f54C3d65f7FBc0aBf5B21E8f",
    "university": "University of Pisa",
    "faculty": "Computer Science",
    "degreeTitle": "MasterDegree",
    "grade": "110/110"
  },
  "proof": {
    "type": "EthereumEip712Signature2021",
    "created": "2026-03-24T18:35:43Z",
    "proofPurpose": "assertionMethod",
    "verificationMethod": "did:ethr:sepolia:0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC#controller",
    "proofValue": "0xbb5139454ddb16428a38c0eb540c00e0cfbdbc67ee01dce1fa3fe8f831211d7e332de77be9971502"
  }
}
```

Mostriamo una VC firmata in cui **l'issuer** University of Pisa, identificato dal DID specificato **certifica** che **l'holder**, identificato dal DID specificato, ha un **master degree** in **Computer Science** con voto **110/110**.

@context: Indica che la VC segue lo standard ufficiale W3C. Deve essere interpretata secondo tali regole.

Type: Indica il tipo di credenziale, in questo caso è sia una VC standard che una VC di tipo UniversityDegreeCredential.

Issuer: Specifica i dati dell'issuer che rilascia e firma la credenziale, in questo caso il suo DID, che è collegato a una chiave pubblica.

issuanceDate: Data di emissione della VC.

credentialSubject: Parte della VC che contiene i dati dell'holder. Viene specificato il DID dell'holder, l'università, la facoltà, il titolo ottenuto e il voto se presente.

proof: Contiene i dati della **firma crittografica** che rende la VC verificabile. Descriviamo i campi contenuti al suo interno:

type: Indica **il tipo di firma crittografica utilizzata**. Serve al verificatore per capire quale algoritmo usare per verificarla. In questo caso la firma è basata su EIP-712.

created: Indica **quando è stata creata la firma**.

proofPurpose: Specifica **perché la firma è stata fatta**. `assertionMethod` significa che l'issuer sta dichiarando che questi dati sono veri.

verificationMethod: Indica **quale chiave pubblica usare per verificare la firma**. In questo caso si indica un DID collegato ad una chiave pubblica.

proofValue: Contiene **la firma digitale vera e propria**.

veramo/types/credential.ts

Modulo che contiene la definizione del modello delle VC e le costanti utilizzate per la loro creazione. Definisce il modello dei dati del Credential Subject, dei dati dell'issuer e dell'intera VC, come specificato sopra. Include costanti e tipi usati per la creazione della VC. Indica quali campi utilizzare per effettuare la firma EIP712. Indica su quali campi effettuare la Selective Disclosure.

veramo/issue-for-dao.ts

Script che emette Verifiable Credential firmate EIP-712 secondo lo schema visto precedentemente. In questo caso l'issuer è un'Università e gli holder sono i membri dell'università che hanno un determinato grado.

Il flusso di esecuzione è il seguente.

Lo script carica da un file `.env` di ambiente la private key dell'issuer che firmerà tutte le VC, e controlla che la private key sia una hex string lunga esattamente 32 byte.

Carica dal file di `env` una sequenza Mnemonic usata per rigenerare i wallet degli holder in modo deterministico.

Legge il file `deployedAddresses.json` contenente gli indirizzi deployati dei contratti DAO, in cui sono presenti gli indirizzi del token, del timelock, del governor, del registry e dell'issuer. Dal JSON estrae l'indirizzo dell'issuer fidato della DAO.

A partire dalla private key ottenuta dall'`env` otteniamo il wallet (address+public+private key) dell'issuer. Confrontiamo che l'indirizzo dell'issuer fidato della DAO corrisponde con l'indirizzo del wallet, poiché la chiave usata per firmare la CV deve coincidere.

Costruiamo il percorso della cartella locale e della cartella condivisa con la DAO dove salvare le VC, e le inizializza in stato pulito, ovvero eliminando eventuali JSON di VC già presenti.

Lo script crea poi il dominio EIP-712, che serve a contestualizzare la firma typed data.

In questo caso **abbiamo scelto** un dominio universale, cioè **non legato** a una chain o a uno specifico smart contract, in modo che la firma può essere verificata ovunque.

Lo script passa poi nella fase vera e propria di emissione delle credenziali.

Lo script itera per tutti gli holder definiti nel file di configurazione. In base alla stringa mnemonic viene generata una master key e da questa vengono derivati i wallet degli holder, in modo deterministico. Poi converte l'address dell'holder ottenuto in DID.

Viene generato il timestamp di emissione a precisione secondi.

Viene poi **costruito il payload** contenente i dati informativi della VC, che devono essere firmati dall'issuer. Sono inclusi il DID dell'issuer, il timestamp di emissione, il DID del holder, il nome dell'universita emittente, la facolta, il titolo di studio e l'eventuale voto.

La VC viene poi firmata con la chiave privata dell'issuer, utilizzando EIP-712.

Viene costruito il JSON VC finale, contenente anche i dati da non firmare e la proof. Il JSON viene salvato sia in una directory locale che in una directory condivisa con la DAO.

```
// Prende i dati della credenziale e li fa firmare crittograficamente al wallet dell'issuer con EIP-712.  
const proofValue = await issuerWallet.signTypedData(domain, VC_TYPES, vcForSigning)
```

Funzionamento firma

IssuerWallet è il wallet in cui è contenuta la chiave privata dell'issuer. Con signTypedData() non viene firmata una semplice stringa libera, ma dei dati strutturati secondo lo standard EIP-712.

Il parametro domain serve a definire il contesto della firma. Il parametro VC_TYPES descrive la forma esatta dei dati da firmare. Dice quali campi da firmare esistono, in che ordine stanno e di che tipo sono. Il parametro vcForSigning, contiene i valori concreti della credenziale, che sono indicati come da firmare.

La funzione costruisce internamente un hash digest, hashando i campi id, university, faculty, degreeTitle e grade, ovvero i campi del credentialSubject. Poi vengono hashati issuer, issuanceDate e credentialSubject. Poi gli hash ottenuti vengono combinati ed hashati insieme.

Poi viene hashato il dominio e la struttura dei tipi da firmare. Poi questi hash vengono combinati e hashati in un unico **digest**.

Questo hash digest viene poi **firmato** dal wallet dell'issuer, con la sua chiave privata, usando EIP-712.

Il risultato finale, ovvero la firma digitale vera e propria, viene salvato in **proofValue**, in formato esadecimale.

Se qualcuno modifica anche solo un campo della credenziale, per esempio il voto o il titolo, la firma non risulterà più valida.

TODOLIST: Aggiornare Modulo Veramo a nuove VC, Rendere credibile modello di skill e topic, introdurre bitmap, scrivere script semplici e chiari, eseguire misurazioni gas, fare deploy su testnet.

5. Verifiable Credentials Integration

Separerei questa parte dal modello perché è uno dei contributi tecnici principali.

5.1 Credential Issuance

Come l'issuer rilascia la credential.

5.2 Trusted Issuers

Come la DAO stabilisce quali issuer sono affidabili.

5.3 Credential Verification

Come viene verificata una VC.

5.4 On-Chain Verification

Perché hai scelto di verificare dentro l'infrastruttura della DAO invece di affidarti a un verifier esterno.

5.5 Credential Expiration and Revocation

Come gestisci:

- credenziali scadute
- credenziali revocate
- aggiornamento delle competenze

5.6 Security Considerations

- credential forgery
- trusted issuer compromise
- replay
- non-transferability

8. Experimental Evaluation

Questo è probabilmente il capitolo più importante insieme al 4.

Io imposterei esperimenti molto semplici ma significativi.

8.1 Experimental Setup

Definisci:

- numero di utenti
- token posseduti
- competence score
- topic
- parametri α e β

8.2 Baseline: Token-Based Governance

Esempio:

User Token Competence

Alice 800 0

Bob 100 100

Carol 100 75

Mostrici cosa succede con governance tradizionale.

8.3 Hybrid Governance

Provi:

- 75% stake / 25% competence
- 50% / 50%
- 25% / 75%

e osservi come cambia il voting power.

8.4 Whale Scenario

Molto importante.

Un utente ha una quantità enorme di token ma competenza bassa.

Valuti quanto il modello riesce a ridurre la dominanza.

8.5 Expert Minority Scenario

Un piccolo gruppo ha pochi token ma alta competenza.

Vedi quanto riesce a incidere.

8.6 Topic-Based Scenario

Uno stesso utente può avere:

- alta competenza in Computer Science
- bassa competenza in Finance

Mostrici che il suo voting power cambia a seconda della proposta.

8.7 Gas Cost

Visto che hai già misurato operazioni come upgradeCompetence, inserirei:

- credential registration
- competence update
- proposal
- vote
- execute

e confronto dei costi aggiuntivi rispetto alla governance tradizionale.

9. Discussion

Qui interpreti i risultati.

- Il modello riduce davvero la concentrazione?
- Quanto pesa la scelta di α e β ?
- Quando la competence-based governance è utile?
- Quando invece può essere controproducente?
- trade-off decentralizzazione / competenza
- centralizzazione degli issuer
- privacy delle credenziali
- Sybil resistance
- governance capture
- scalabilità

Qui puoi anche fare una considerazione importante:

non stai cercando di sostituire completamente il capitale con la competenza, ma di bilanciare skin in the game ed expertise.

10. Limitations and Future Work

- Zero-Knowledge Proofs per non esporre le credenziali
- multiple credentials
- competenze con scadenza
- reputation dinamica
- issuer decentralizzati
- sistemi multi-topic
- pesi dinamici

- quadratic voting + competence
 - delegazione verso esperti
 - cross-chain identity
-

11. Conclusion

Breve:

- problema iniziale
- soluzione proposta
- implementazione
- risultati
- contributo

3. Related Work

Qui confronti la tua soluzione con ciò che esiste.

- Token-weighted voting
- Quadratic voting
- Reputation-based governance
- Identity-based governance
- Soulbound Tokens
- VC applicate alla blockchain
- Expertise/reputation-based DAO governance

Alla fine metterei una sezione:

3.1 Research Gap

Dove dici chiaramente che il tuo lavoro mira a combinare:

stake + competenze certificate + rilevanza della competenza rispetto alla proposta.